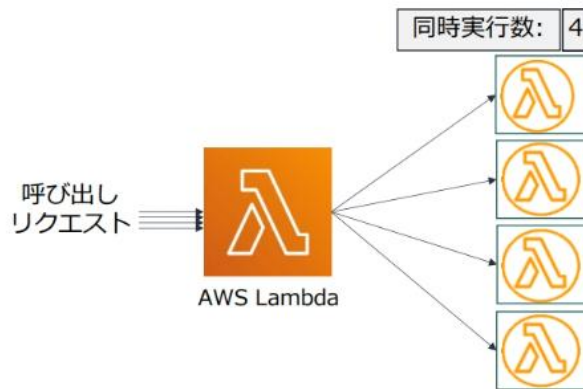


07 : 優れた Lambda 関数を書く

Chapter 7

■ 同時実行数: 同時に実行されている関数呼び出しの数



Lambda 関数の呼び出しは、それぞれ一時的な環境（オンデマンドでスピンアップされ、短時間持続してから停止される環境）で実行されます。Lambda では、こうした関数の環境の作成と破棄を管理します。これらの環境を直接制御することはできません。

同時実行数は同時に実行されている Lambda 関数呼び出しの数です。Lambda は、リクエストを受信したときにオンデマンドで環境を提供します。軽食レストランの料理人の目から見ると、同時に点火しているガスレンジの数のようなものです。これは Lambda で迅速に水平スケーリングを実現する方法です。

Lambda が使用する同時実行数は、複数の要因によって決定されます。その一部はユーザーが制御できませんが、サーバーレスのデベロッパーとしてはこれらすべてを認識する必要があります。

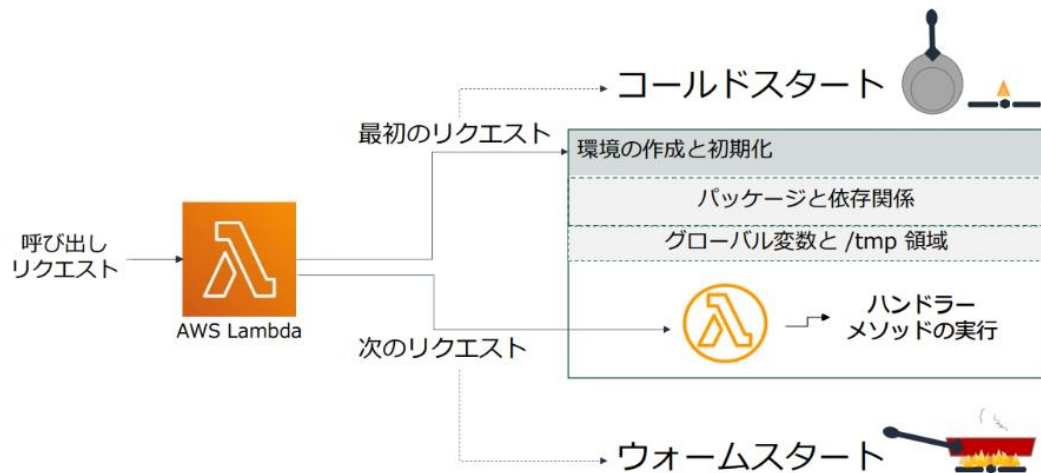
同時実行数に影響する要因は次のとおりです。

- ・関数での予約された同時実行数: これは、その関数に対してそのリージョンのクォータのサブセットを予約し、関数で許可される最大同時インスタンス数も規定する、関数ごとのオプション値のセットです。
- ・同時実行数（リージョンのクォータ）: リージョン別アカウント内のすべての Lambda 関数で同時に実行できる呼び出し数の合計です。AWS では、これがアカウントのソフトクォータとして設定されます。
- ・関数の同時実行スケーリングリミット（リージョンごと、関数ごと）: Lambda は、非常に短時間に大量のリクエストが発生した場合に、オーバースケーリングを防ぐためにこれを使用します。Lambda は、アカウントが同時実行数（リージョンのクォータ）に達するまで、これらのリクエストを処理するために実行環境の数を自動的にスケールアップします。この制限を変更することはできません。
- ・リクエストレートと関数実行時間: これは、関数の新規呼び出しリクエストが到着する速度と、Lambda で関数を実行するのにかかる時間を示します。

・ イベントソース:

- Lambda は、同期および非同期イベントソースのリクエストに対応するために、適用可能なクォータまで同時実行数を引き上げます。リクエストレートに平均関数実行時間を乗算して、関数が使用する同時実行数を決定します。
- Amazon Simple Queue Service (Amazon SQS) がイベントソースの場合、Lambda はキューの長さとエラー率に基づいて、ポーリングレート/同時実行数を増減します。Lambda は、SQS キューからメッセージを取得するために使用する各並列プロセスに対して 1 つの同時呼び出しを使用します。デフォルトは 5 つの同時呼び出しから始まります。
- ストリーミングソースでは、ストリーム上のシャードの数と関数の並列化の設定に基づいて同時実行数が設定されます。

■ Lambda で新しい「コールド」環境の作成や「ウォーム」環境の再利用が可能に



初めて関数が呼び出されると、Lambda ではその関数が実行される実行環境が作成されます。Lambda ではコードが環境にダウンロードされ、ランタイムだけでなく、関数に含まれるパッケージ、依存関係、グローバル変数も初期化されます。この環境では関数の一時領域 (/tmp) も初期化されます。環境の準備ができると、Lambda は Handler メソッドから始まる関数を実行します。

その呼び出しが完了すると、Lambda はその初期化された環境を再利用して、次のリクエストを実行できます (次のリクエストを最初のリクエストの直後に受信した場合)。したがって、2 番目のリクエストでは、初期化手順はすべてスキップされ、前の実行時に設定された /tmp 領域とグローバル変数を使用して、関数のハンドラーメソッドが直接実行されます。

この例では、最初のリクエストは「コールド」スタートになり、2 番目のリクエストは「ウォーム」スタートになります。

朝誰かが卵を注文すると、料理人はガスレンジに点火して、卵を調理する前に冷たい平鍋を加熱しなければなりません。

- 最初の注文が終わった直後に別の注文が入ると、料理人はもう温かくなっている平鍋を使って 2 番目の注文を調理することができます。料理人は平鍋が温まるのを待つ必要がなかったため、最初の卵よりも早く卵が用意できます。
 - 関数がしばらく実行されなかった場合、コールドスタートが発生する可能性があります。Lambda では、追加のリクエストが受信されなければ環境は「停止」されます (午後 2 時になって 1 時間前から注文が入って来ないので、料理人はガスレンジの火を消します)。
 - コールドスタートは、Lambda がリクエストに追いつくために同時実行数を引き上げる必要がある場合にも発生します (料理人は、最初の調理が終わる前に別の卵の注文があるので、2 番目の平鍋を用意してガスレンジに点火する必要があります)。
- 図の例では、2 番目のリクエストが開始される前に最初の呼び出しが完了するため、同時実行は 1 です。しかし、最初のリクエストがまだ実行されている間に 2 番目のリクエストが入って来た場合は、Lambda は同時実行を 2 にして 2 番目の環境をスピンアップし、両方ともコールドスタートとなります。

安定した予測可能なリクエストレートであれば、コールドスタートなしで長時間続けることができます。新しいリクエストは、以前のリクエストで使用されたばかりの環境に転送されます。しかし、リクエストレートが急上昇する傾向があったり、予測できないトラフィックが発生しやすかったりする場合は、コールドスタートを受け入れるか、コールドスタートの可能性を軽減する必要があります。

このモデルでもう 1 つ想定される課題は、一時的な環境の中で関数が必要とするすべてのものを提供することです。一時的な環境では一時領域が限られており、外部リソースを初期化の一部として環境にロードしていなければ外部リソースを使用できないためです。

■ Lambda 関数のベストプラクティス

- 環境の再利用を活用する
- バックグラウンドプロセスやコールバックが完了するまで待機する
- データベースプロキシを使用してデータベース接続プーリングを管理する
- 状態データを外部で維持する
- デプロイパッケージのサイズと依存関係の複雑さを最小限に抑える
- 大規模なアセットまたは共有アセット用にAmazon EFS をマウントする
- プロビジョニングされた同時実行数を使用してコールドスタートを回避する

このモデルは、サーバーレスを検討し始めている開発者のために、新しい案をいくつか提示します。

必要なこと:

- ・一時環境のコード: コードが実行される環境に直接アクセスすることはできません。Lambda は新しい環境の作成と破棄を管理します。例えば、呼び出し A をウォーム環境に直接転送することはできません。
- ・ステートレス関数の作成: 各 Lambda 関数は独自の環境で実行されるため、関数の呼び出し間で直接情報を共有することはできません。
- ・コードの初期化と実行の速度を最適化し、コールドスタートの影響を最小限に抑える: 関数リソース開始のサイズと速度はサーバーベースのセットアップよりもはるかに重要です。関数をできるだけ迅速に実行できるように、新しい環境の起動時間をできるだけ短くする必要があります。これは、コストとパフォーマンスの両方で重要となります。

このような特性に基づいて関数に適用すべき 6 つのベストプラクティス:

1. すでに存在するリソースを再初期化する時間を無駄にしないように、ウォームスタートを活用するための関数を作成します。ただし、新しい呼び出しが入って来て前の呼び出しから次の呼び出しに移るときに、前のプロセスを実行したままにしないでください。ウォーム環境では前のプロセスの実行が続いているためです。
2. データベース接続を管理するようにデータベースプロキシを設定します。
3. Amazon DynamoDB などの外部データストアを使用して、ステートデータを永続化して、関数をステートレスにします。
4. 関数をできるだけ早く初期化できるように、デプロイパッケージはできるだけ小さくし、依存関係を制限します。
5. /tmp 領域の制限に収まらない大きなバイナリと連動する必要がある関数には、Amazon Elastic File System (Amazon EFS) ストレージをアタッチします。また、大規模なライブラリやその他のリソースを実行環境に読み込むことができなかったり、現実的でなかったりする場合は、Amazon EFS を使用して呼び出し間でライブラリやリソースを共有することもできます。
6. コールドスタートを減らすか、できる限り無くすには、プロビジョニングされた同時実行数と呼ばれる機能を使用して、必要な数の環境を「常にウォーム」状態に保ちます。

■ 環境の再利用の例: ハンドラーの外部のグローバル変数を初期化する

```
// 項目を追加するためにクエリを行うクライアントを作成する
const AWS = require("aws-sdk");
var ddbClient = new AWS.DynamoDB();

// 環境変数から DynamoDB のテーブル名を取得する
const tableName = process.env.SAMPLE_TABLE;
exports.putItemHandler = async (event) => {

  const { body, httpMethod, path } = event;
  if (httpMethod !== 'POST') {
    throw new Error(`postMethod only accepts POST method, you tried: ${httpMethod} method.`);
  }
  // snip
};
```

ハンドラーの外部で
DynamoDB クライアントを
宣言する

環境の再利用の一例は、グローバルをハンドラーの外部に配置することです。

コードで外部設置された設定内容や依存関係を取得する場合、最初の実行後はローカルに保存し、そこから参照するようにします。変数やオブジェクトを呼び出すたびに再初期化することを制限します。(ハンドラーコードの外部の) Lambda 関数コード内の宣言は、関数が再度呼び出されたときに初期化されたままとなります。

接続を作成する前に、既存の接続の存在をチェックするロジックをコードに追加する必要もあります。接続が存在する場合は、既存の接続を再利用します。

この Node.js の例では、DynamoDB クライアントはハンドラーの外部で宣言され、現在の呼び出しだけでなく、その環境に転送された呼び出しがまだウォームであれば、その呼び出しにも使用できます。

環境の再利用の例: 一時キャッシュとして tmp スペースを使用

```
const fs = require('fs');
const AWS = require('aws-sdk');
const s3 = new AWS.S3();

exports.handler = async (event, context) => {
  let params = { Bucket: 'myprivatebucket', Key: 'config.json' };
  let config;
  try {
    config = fs.readFileSync('/tmp/config.json');
  }
  catch (e) {
    console.error(e);
    let data = await s3.getObject(params).promise();
    config = data.Body.toString('utf8');
    //write to /tmp/config.json
    fs.createWriteStream('/tmp/config.json').write(config);
  }

  return;
};
```

Node.js 16.x CommonJS
AWS-SDK V2

/tmp/config.json から構成をロード

S3 bucket から構成をダウンロード

環境の再利用を利用するもう 1 つの方法は、ローカル キャッシュに保存したデータがあるかどうかを確認するコードを追加することです。各呼び出し環境では、環境内に残る /tmp ディレクトリ内に少量のディスク領域が提供されます。

これは、関数が tmp ディレクトリで config.json を検索する Node.js の例です。これがウォーム環境の場合、ファイルを再ロードする必要はありません。tmp ディレクトリ内のファイルの検索でエラーが発生した場合、関数はそのファイルをロードします。

■ 例: コールバックが完了するのを待つ

// Callback の使用

```
exports.handler = function (event, context, callback) {  
  dbClient.listTables(null, callback);  
};
```

完了をチェックする
Node.js オプション

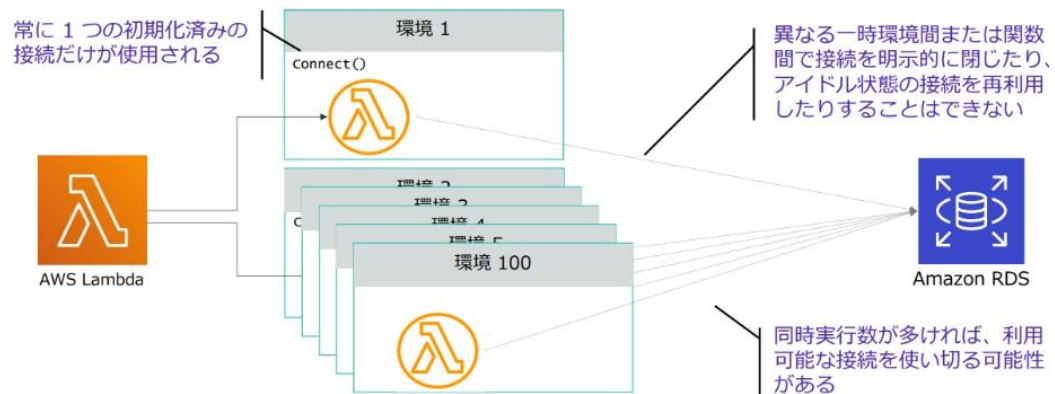
// Async/Await の使用

```
exports.handler = async function (event, context) {  
  const tableNames = await dbClient.listTables().promise();  
  return tableNames;  
};
```

バックグラウンドプロセスが終了するまで待たずに関数が終了すると、その環境が存在している間、バックグラウンドプロセスは引き続き実行されます。このためそのウォーム環境を取得する後続の呼び出しが、前回の呼び出しで実行を開始したプロセスによって想定外の結果となる可能性があります。選択した言語の言語要素を使用して完了をチェックします。

■ データベースプロキシを使用して接続プーリングを管理する

データベース接続を管理するための他の手法の問題



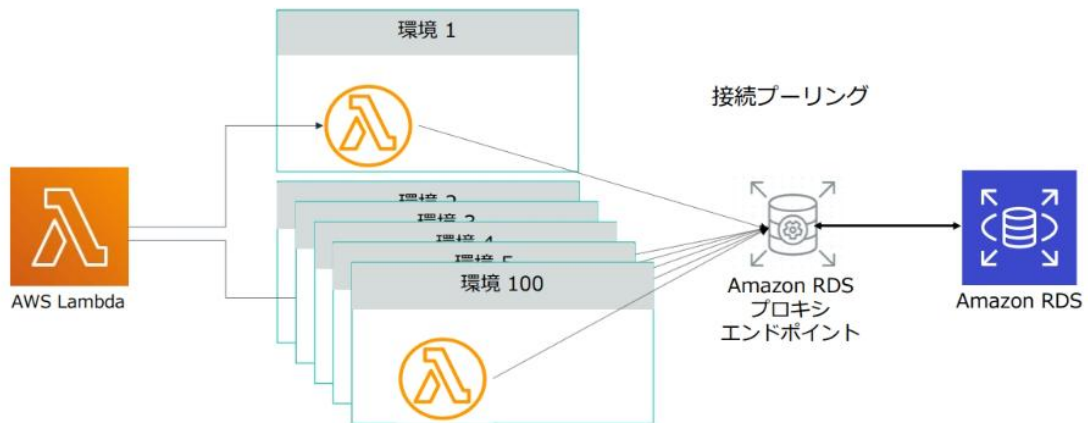
データベース接続を管理するために使用できる手法は、Lambda ライフサイクルとその一時環境に適用すると価値が限定されます。

例えば、起動時に複数の接続を初期化して、アプリケーションで使うことができます。Lambda を使用すると、ハンドラーの外部で接続を初期化してからコード内でそのチェックを実施し、その環境でウォームスタートを行う後続の呼び出しで使えるようになります。ただし、Lambda はその環境内で同時に複数の呼び出しを実行することはありません。したがって、関数の初期化の一環として、複数の接続をオープンする理由はありません。

別の課題: 環境がリサイクルされたときに明示的に接続を閉じることができないことです。これは、Lambda 環境の破壊を示すフックがないためです。接続をクリーンアップするフォールバックとしてデータベースの存続時間 (TTL) を使用できますが、それでもセッションリークが発生する可能性があります。呼び出し環境のライフサイクルを制御できないため、接続がアイドル状態になる可能性があります。また、環境を 2 つの異なる Lambda 関数で共有できないため、関数間でアイドル状態の接続を再利用できません。

Lambda の同時実行数が、データベース接続数の許容量以上に増加する可能性もあります。接続数の許容量と一致するように、同時実行数を制限する方法を見つける必要があります。関数レベルで予約された同時実行数を使用して、Lambda が作成する可能性がある接続の数を制限できますが、特に異なるアプリケーションへの接続を分離する必要がある場合は、適切なレベルの設定を想定するのが困難な場合があります。

データベースプロキシは接続を処理して関数に単一エンドポイントを提供する



データベースプロキシでは、他のメリットに加えて、接続プーリングを提供できます。これにより、関数は Amazon Relational Database Service (Amazon RDS) のプロキシエンドポイントとのやりとりが可能となり、データベースへの着信接続数は、必要に応じてプロキシで管理されます。これは、高レベルの同時実行数が予想される関数では、はるかに効率的です。AWS では、Amazon RDS MySQL や Postgres だけでなく Amazon Aurora でもデータベースプロキシを提供しています。

他の機能と同様に、ワークロードに対するコストとメリットを比較します。データベース接続を獲得するために競合する高レベルの同時実行数が得られない場合は、前のスライドに示した方法（接続を初期化して再利用し、予約された同時実行数で接続を制限し、TTL 設定によりアイドル状態の接続を閉じる）を使用できます。

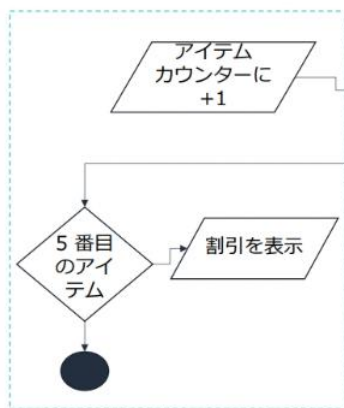
データベースプロキシを使用する場合は、プロキシで使用する接続の割合を設定します。プロキシはデータベースをチェックして、設定されている最大接続数を確認します。例えば、データベースが 200 の接続を処理するように設定されている場合、割合を調整しなければ、プロキシはすべての接続を使用します。プロキシは、Amazon RDS に対して開く接続の数をスケールします。接続数は複数になる可能性があります。ただし、Lambda 関数に対して複数になることはありません。

プールにあらゆることの負荷をかけないようにします。管理など、他のアクティビティの分を残しておく必要があります。使用しているバックエンドの種類について考えてみてください。データベースが他のアプリケーションと共有されている場合は、そのアプリケーションが必要とする接続数を考慮する必要があります。

また、複数のプロキシを設定することもできます。例えば、個別の認証情報を使用したり、ユースケースとのバランスを取ることもできます。

理にかなった有効な接続数の割合を把握します。モニタリングに関する主な考慮事項には、Amazon RDS プロキシメトリクス、データベース接続、クライアント接続などがあります。

■ 状態データを外部で維持する



例

- Amazon DynamoDB は、数ミリ秒単位のレイテンシーと水平スケーリングを実現
- VPC 内の Amazon ElastiCache
- 大規模オブジェクトの中間ストレージ用の Amazon S3
- 長時間実行中の ETL プロセスの状態を保存するための Amazon EFS

前述のように、ウォームスタートが利用できる場合はウォームスタートを活用しますが、ウォームスタートが利用できるとは想定できません。したがって、/tmp ディレクトリに書き込んだデータがそのまま使用できると想定することはできません。

例えば、アイテムをショッピングカートに追加し、カート内のアイテムカウンターをインクリメントする機能があるとして、そのカウンターの値をローカルに保存しても、次の呼び出しでコールドスタートになると、ローカルのカウンターはリセットされ、状態を適切に維持できなくなります。状態を維持する必要がある場合は、外部ストアを使用してデータを保持します。

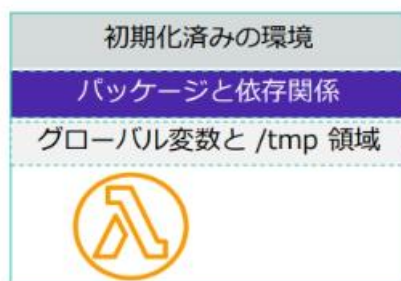
DynamoDB はサーバーレスで、水平スケーリングで Lambda 呼び出しを処理します。また、DynamoDB のレイテンシーは数ミリ秒であるため、状態情報の保存に最適です。

VPC に Lambda 関数を配置する必要がある場合は、Amazon ElastiCache の使用を検討してください。Amazon ElastiCache は DynamoDB よりも費用負担が少なく、ソート済みセットなどの機能を使用できます。

Amazon Simple Storage Service (Amazon S3) は、関数が使用する大規模オブジェクト（一連のステップで処理するイメージの中間ストレージなど）を低コストで持続する方法として使用できます。Amazon EFS を使用すると、状態を保存して関数呼び出し同士（長時間実行される ETL プロセスなど）で共有できます。

■ デプロイパッケージのサイズと依存関係の複雑さを最小限に抑える

- ランタイムに必要なもののみに限定する
- 簡素化されたフレームワークが望ましい



関数のコードは、スクリプト (Python) またはコンパイルされたプログラム (Java、C#) とその依存関係から成ります。Lambda のコンソールまたはツールキットで関数を作成する場合は、クライアントはコードの ZIP アーカイブを作成します。これがデプロイパッケージです。Lambda API、コマンドラインツール、または SDK を使用して関数を管理する場合は、デプロイパッケージを作成します。

コンパイル言語のデプロイパッケージを手動で作成し、関数に依存関係を追加する必要があります。

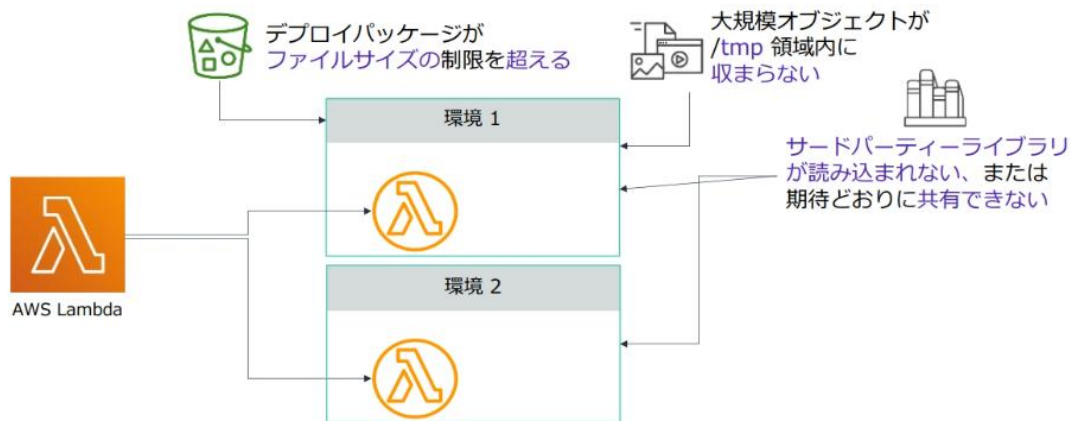
関数を呼び出すと、デプロイパッケージがダウンロード、解凍され、呼び出し環境にロードされます。パッケージを最小化するとコールドスタートが短縮され、Lambda が環境内のランタイムを初期化する時点から課金されるため、コストが削減されます。

デプロイパッケージのサイズを必須ランタイムのみに最小化します。これにより、呼び出しに先立ってデプロイパッケージをダウンロードし、解凍にかかる時間を短縮できます。これは、コンパイル言語で作成された関数では特に重要です。

AWS SDK ライブラリ全体をデプロイパッケージに入れることは避けてください。その代わりに、必要な SDK コンポーネントを取り出せるモジュールを選び出すようにします (DynamoDB、Amazon S3 SDK モジュールや Lambda の主要なライブラリなど)。

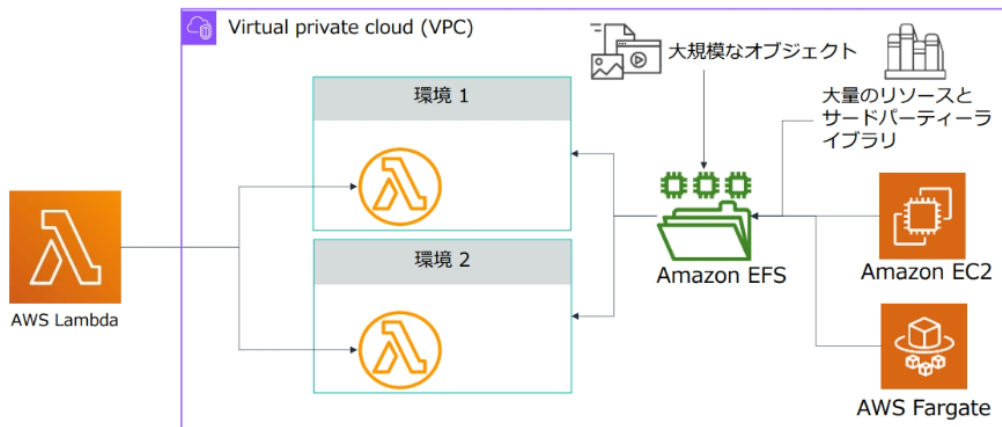
■ 大規模なアセットまたは共有アセット用に Amazon EFS をマウントする

■ Lambda 環境の課題が生じるシナリオ



Lambda 呼び出し環境のサイズと性質により、非常に難しくなるユースケースもあります。例えば、デプロイパッケージのサイズ制限を満たすほど小さくできない場合や、一時領域にロードできないメディアファイルや機械言語モデルなどの非常に大きなオブジェクトを扱う必要がある場合などです。サーバーから実行した場合と同じように、Lambda 呼び出し環境にロードして共有できないサードパーティライブラリもあります。また、前に述べたように、データを直接共有することはできません。

■ 大容量ファイル、外部ライブラリ、ファイル共有に Amazon EFS ボリュームを使用する



呼び出しは Amazon EFS に保存でき、保存されたデータは共有できる

Amazon EFS ボリュームをマウントすることで、前のスライドで示した課題に対処できます。Amazon EFS に大規模オブジェクトやサードパーティライブラリを保存して、Lambda 関数からアクセスできます。

Amazon EFS を使用するともう一つの利点があります。Amazon EFS にデータを書き込んで、呼び出し間で共有できるのです。これにより、Amazon EFS を使用すると、1 回の呼び出しで更新されたデータを、次の呼び出しで続けて使用できます。Amazon EFS に書き込むのに、特別なコーディングは必要ありません。ブロックストレージにファイルを書き込むのと同じコマンドを使用するだけです。

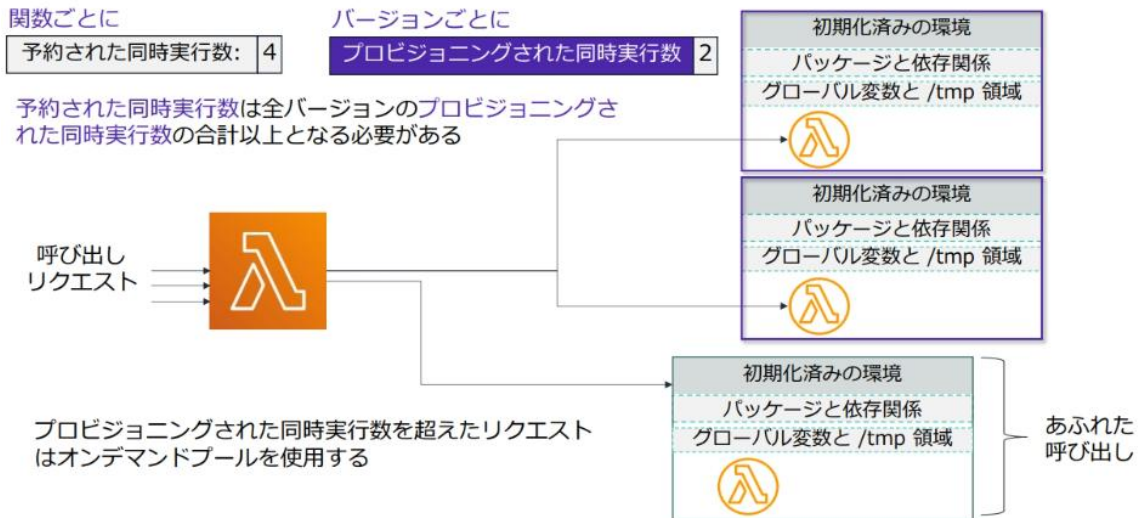
他のソースを使用して、Amazon EFS のデータだけでなく、AWS Fargate や Amazon Elastic Compute Cloud [Amazon EC2] などでも更新できます。これにより、ハイブリッドシナリオがはるかに簡単になり、Lambda で実行できるワークロードが増えます。

Amazon EFS ボリュームをアタッチして呼び出し間で共有する機能を使用すると、アーキテクチャが簡素化され、ワークロードの処理コストを削減できます。例えば、ブロックストレージにアクセスできる場合は、データを操作したり動画を処理したりするのに、非常に大きなファイル进行处理の方がはるかに簡単です。Amazon S3 のようなオブジェクトストアではなく Amazon EFS を使用の方が、並列処理のためにファイルの一部を分割するのが簡単になります。また、コンピューティングソースを組み合わせることもできます。例えば、Fargate を使用してファイルを分割してから、Lambda に処理を完了させます。

人工知能 (AI) と機械学習 (ML) のワークロードでは、トレーニング対象のモデルは大規模で、Lambda のメモリまたは /tmp 領域に収まる場合でも、ダウンロード時は実行時間がかかります。Amazon EFS を使用すると、時間を短縮できるのではるかに大きなパッケージにアクセスできます。

Amazon EFS を使用しないリアルタイムアプリの場合、一般的なワークフローでは、Amazon S3 にデータを保存し、ElastiCache に移動して読み込みアクセスを高速化することになります。このパターンは機能しますが、Amazon S3 および ElastiCache ストレージを Amazon EFS ストレージに置き換えることで、複雑さが少なくなり、コストが削減されます。

■ プロビジョニングされた同時実行数を使用してコールドスタートを回避する



関数の予約された同時実行数の一部を、オンデマンド同時実行数ではなく、プロビジョニングされた同時実行数として設定できます。Lambda が設定した環境の数を初期化すると、関数がしばらく呼び出されていなくても、環境は常に「ウォーム」になります。

軽食レストランに例えると、これは将来の注文を見越して、ストーブの上でいくつかの平鍋を温めておくようなものです。

この例では、プロビジョニングされた同時実行数が 2 に設定されている場合、その関数の最初の 2 つの呼び出しは両方ともウォームスタートになります。

プロビジョニングされた同時実行数によって、アプリケーションのコールドスタートのレイテンシーを処理する負担を軽減できますが、環境のプロビジョニングを維持するにはコストがかかります。したがって、作業負荷と関数のためにウォームスタートを維持する重要性の適切なバランスを見つける必要があります。

Lambda から、設定したプロビジョニングされた同時実行数よりも多くの呼び出しのリクエストがあった場合、それらのリクエストはオンデマンドプールに送られ、前述のとおり処理されます。環境が利用可能であれば、リクエストはウォームスタートになる場合があります、またコールドスタートになる場合もあります。

プロビジョニングされた同時実行数が 2 の場合、関数には常に 2 つのプロビジョニング済みの環境が存在することになります。

最初の 2 つの呼び出しが完了する前に 3 番目のリクエストが入ってくると、3 番目の同時実行環境がスピンアップされます。新しい呼び出しがこれですべてだと仮定すると、リクエストはコールドスタートになります。3 番目の要求は、オンデマンド環境で呼び出されます。

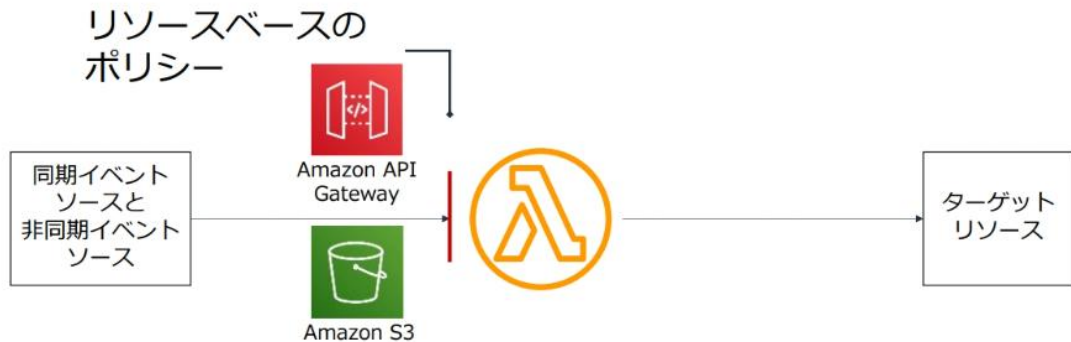
オンデマンド環境に転送されたこれらの呼び出しは、スピルオーバーメトリクス

ProvisionedConcurrencySpilloverInvocations に反映されます。

Lambda で、最初の 4 つの呼び出しリクエストがまだ実行されている間に 4 つ以上の呼び出しリクエストを受信した場合、追加のリクエストは予約済みの同時実行数 4 に基づいて調整されます。

■ 関数を設定する
～アクセス許可、ソース、ターゲット～

リソースベースのポリシーにより関数の呼び出しにアクセス許可が付与される



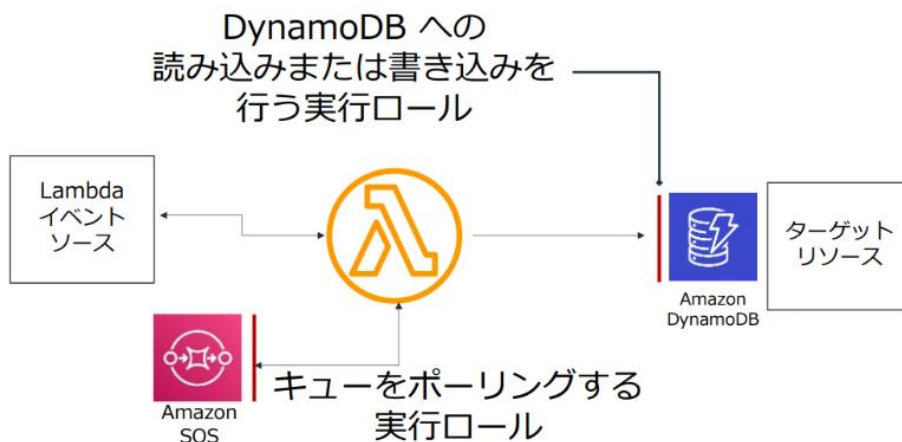
関数にトリガーとターゲットを追加する場合は、次の 2 つの主要なアクセス許可を管理する必要があります。

- 1) 実行ロールを使用して Lambda 関数が他のリソースを操作するためのアクセス許可
- 2) 関数を呼び出すためのアクセス許可（リソースベースのポリシーで決定される）

Lambda を同期または非同期でトリガーするサービスの場合、Lambda コンソールで関数にトリガーを追加すると、サービスで関数が呼び出せるように関数のリソースベースのポリシーがコンソールで更新されます。同様に、別のサービスコンソールから Lambda との統合を設定すると、関数を呼び出すアクセス許可が自動的に作成されます。例えば、Lambda 関数を対象とする Amazon API Gateway で API を作成した場合、その API を呼び出すためのアクセス許可は、API の作成時に追加されます。

Lambda コンソールで使用できない他のアカウントやサービスにアクセス許可を付与する必要がある場合は、AWS CLI を使用してリソースベースのポリシーを更新できます。AWS Identity and Access Management (IAM) ロールを使用して、呼び出しのアクセス許可を付与することもできます。重要なのは、デベロッパーとして関数をトリガーするために、ソースから関数へのアクセス許可を付与する必要があることです。

■ 実行ロールはリソースを操作するための関数のアクセス許可を定義する

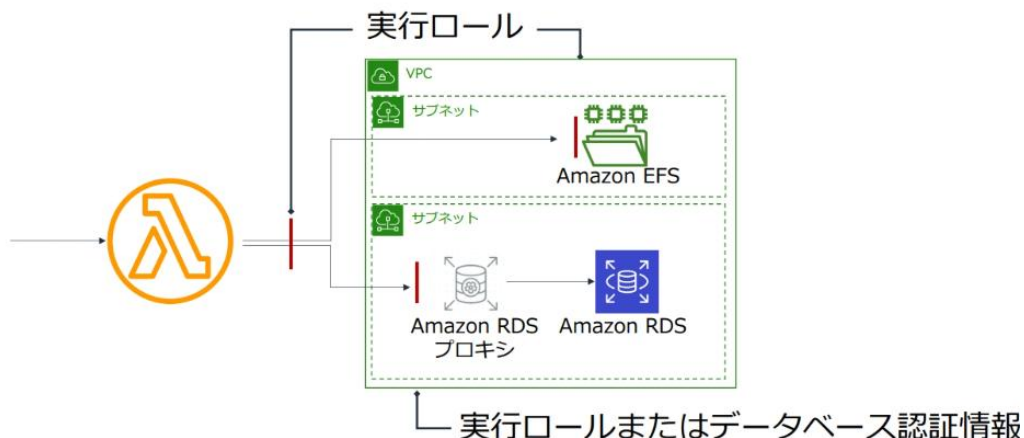


実行ロールは、関数を呼び出すときに Lambda が許可されて引き受ける IAM ロールです。関数の作成時に実行ロールを選択または作成する必要がありますが、IAM を使用してロールに関連付けられたポリシーを変更できます。デフォルトの Lambda 実行ロールには、Amazon CloudWatch Logs への書き込み権限が含まれます。

ターゲットを追加する場合（例えば、データベースへの書き込みや Amazon Simple Notification Service [Amazon SNS] トピックの送信）、Lambda 関数の実行ロールでは、これらのターゲットを操作するためのアクセス許可を必要とします。ロールには、データベースや SNS トピックなどへの書き込み権限が必要です。IAM には、必要なアクセス許可の取得を簡素化するために、関数に追加できるマネージドポリシーが用意されています。

実行ロールは、使用しているイベントソースが Lambda でキューまたはストリームからメッセージを取得する必要がある場合にも関与します。この場合、ストリームまたはキューがイベントソースですが、Lambda はキューまたはストリームをアクティブにポーリングして、レコードのバッチを取得します。Lambda はソースを操作して、キューからレコードを削除したり、ストリーム上でポインタを移動したりする必要があります。したがって、これらのイベントソースタイプでは、実行ロールにはイベントソースに関連するアクセス許可が必要となります。

関数を VPC ターゲットに接続するには実行ロールのアクセス権限が必要



VPC 内のリソースを操作するには、実行ロールにリソースを操作するアクセス許可を付与するだけでなく、関数を VPC に接続する必要があります。実行ロールのアクセス許可には、ネットワークインターフェイスを操作する機能が含まれている必要があります。関数を VPC に接続すると、Lambda によって、関数の VPC 設定内のセキュリティグループとサブネットの組み合わせごとに Elastic Network Interface が作成されます。マネージドポリシーには、必要なアクセス許可が含まれます。

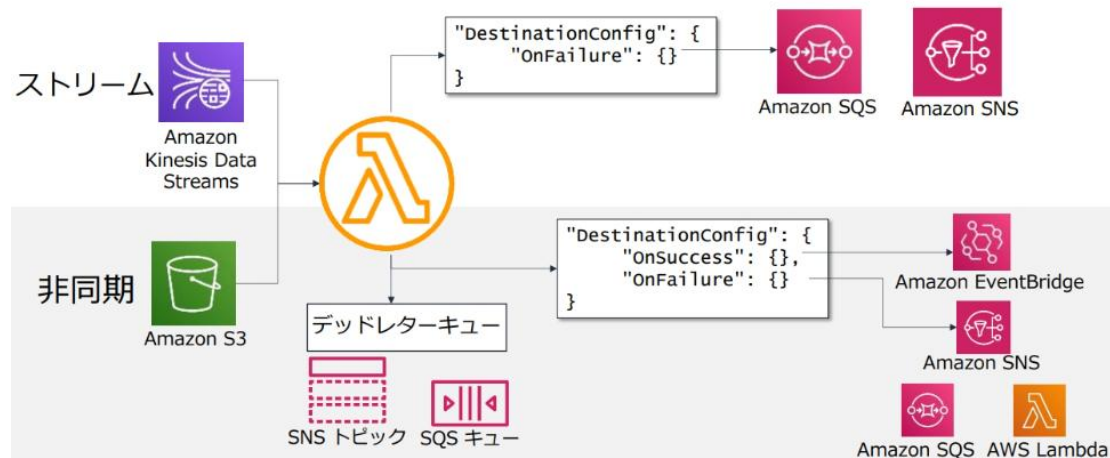
関数の VPC 接続を設定しているユーザーの IAM ロールには、[VPC アクセス許可] ページに記載されているアクセス許可が必要です。

Lambda コンソールには、VPC に接続するためのオプションがあります。設定が完了したら、関数の Amazon EFS ボリュームの設定や、データベースプロキシ機能の設定が可能になります。これにより、Amazon RDS インスタンスへの接続プールを作成できます。関数を適切な VPC やサブネットに接続するまで、これらのオプションの設定は完了しません。

Amazon EFS を使用するには、関数の実行ロールに Amazon EFS を操作するためのアクセス許可が必要となります。

Amazon RDS プロキシを使用する場合は、実行ロールに認証の選択を設定でき、Lambda コンソールによって実行ロールに必要なアクセス許可が追加されます。また、関数がデータベースへの接続に使用するのと同じユーザ名とパスワードを使用することもできます。この 2 番目の方法の欠点は、安全な環境変数で設定するか、AWS Secrets Manager から取得することで、パスワードを関数コードに公開する必要があります。

■ ストリーム呼び出しまたは非同期呼び出しの対象となる送信先



送信先は、関数の呼び出しレコードを受け取る AWS リソースです。実行ロールにより送信先へのアクセス許可を関数に付与する必要があります。

ストリーミングイベントソースでは、OnFailure 送信先をストリームのエラー処理機能と組み合わせて使用して、エラーとなったレコードをストリームから移動し、Amazon SNS や Amazon SQS などのサービスに送信してオフラインで処理できます。

非同期レコードには、OnSuccess と OnFailure の両方の送信先を指定できます。

OnSuccess 送信先を使用して、Amazon EventBridge または Amazon SNS などのサービスや、別の Lambda 関数をターゲットにします。OnSuccess 送信先は、レコードが正常に処理（エラーなしで完了）された場合、呼び出しレコードを受信します。

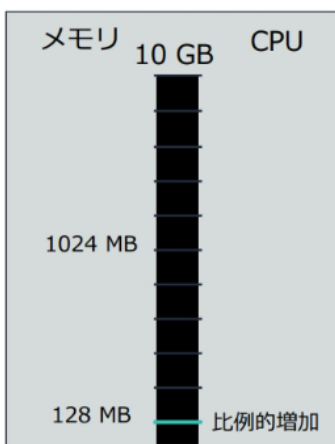
OnFailure 送信先は、処理をリトライしてもエラーとなった呼び出しレコードを受信します。エラーとなったレコードを処理するために、OnFailure 送信先として Amazon EventBridge、Amazon SNS、Amazon SQS などのサービスを設定できます。

OnFailure 送信先に代わる方法として、デッドレターキューを使用して関数を設定し、破棄されたイベントを保存してさらに処理できます。デッドレターキューは、イベントが処理を試行してすべてエラーとなった場合や処理されずに期限切れになった場合に使用されるという点で、OnFailure 送信先と同じように動作します。ただし、デッドレターキューは関数のバージョン固有の設定の一部であるため、バージョンを公開するとロックされます。もう 1 つの違いは、デッドレターキューでは、Lambda はレスポンスの詳細情報を付けずにイベントの内容のみを送信することです。送信先では、Lambda は呼び出しレコードを送信します。呼び出しレコードには、リクエストとレスポンスの両方の詳細情報が含まれます。

デッドレターキューは、SNS トピックまたは SQS キューのいずれかが可能です。関数の実行ロールには、メッセージをキューに送信したり、トピックにメッセージを公開したりするアクセス許可が必要です。

■ メモリとタイムアウト

メモリとタイムアウトの選択



- **メモリに比例して CPU の使用量が決まる**
 - メモリが大きいほどコストが高くなる
- タイムアウトは、正常終了と処理がスタックした関数に支払いを行わないことのバランスを取る
 - 期間が長いほどコストが高くなる
- **メモリが大きいほど持続時間が短くなる場合があります、全体的なコストが下がる可能性がある**

Lambda 関数の基本設定のメモリ設定では、関数に割り当てるメモリの量を Lambda に指定します。この設定は、関数で利用できる CPU パワーの量も決定します。メモリサイズは CPU に正比例します。例えば、256 MB のメモリを持つ環境では、1 セット 128 MB の容量の 2 倍になります。

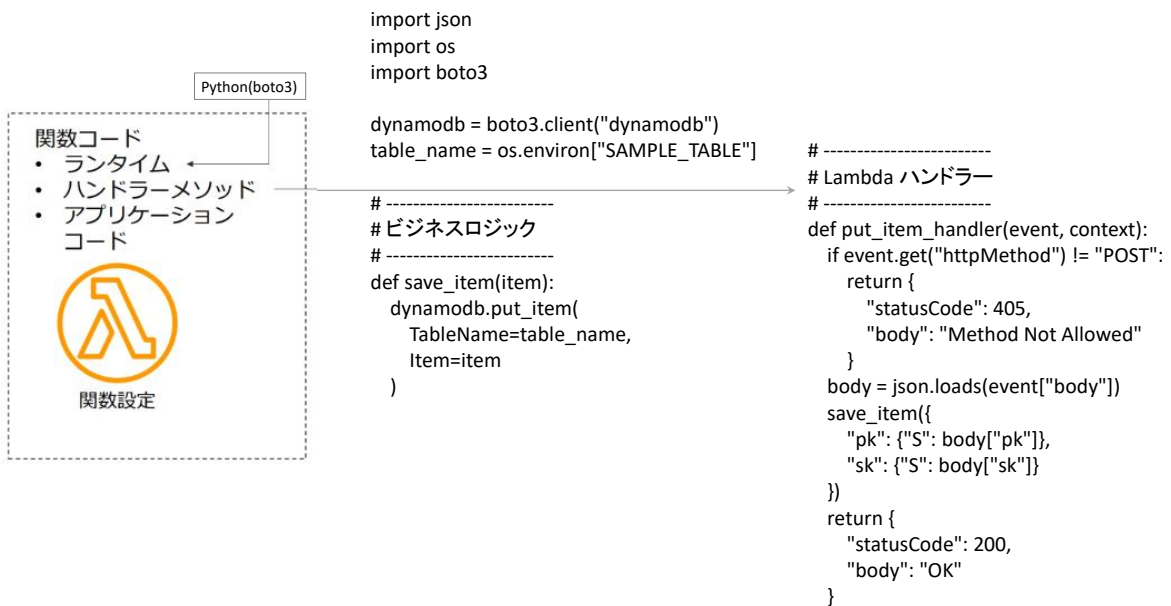
関数が処理を終了するのに十分な時間と、呼び出しで応答が停止した場合に Lambda が待機する時間の制限のバランスが適切なタイムアウト値を選択します。関数がタイムアウトになって処理が完了していない場合は、Lambda によって強制的に停止されます。

関数のコストはメモリの割り当てと持続時間に関係しているため、シナリオに応じて関数のパフォーマンスを最適化するスイートスポットを見つける必要があります。

■ 関数コード

Lambda ライフサイクルに関するベストプラクティスだけでなく、関数のコーディングのベストプラクティスにも従ってください。

■ 実際の関数はハンドラーメソッドで開始され、選択したランタイムで実行される



関数自体には、あらゆるアプリケーションコードが存在します。関数の設定の一部としてランタイムを選択すると、Lambda は環境を初期化するときにそのランタイムをロードします。

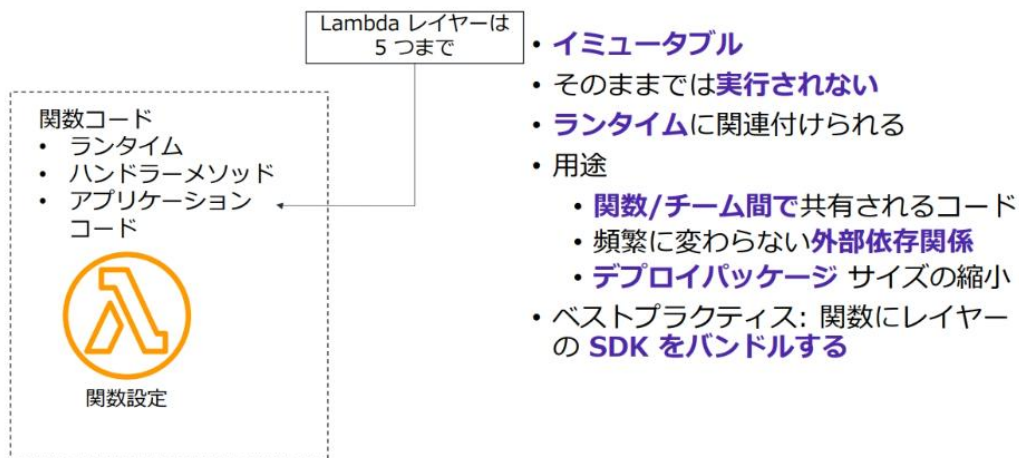
ベストプラクティスは、Lambda ハンドラーをビジネスロジックから分離することです。

サンプルコードでは、Lambda ハンドラーは `put_item_handler(event, context)` として定義されており、この関数は イベントの受け取り、HTTP メソッドの判定、レスポンスの返却 といった Lambda 固有の責務のみを担っています。

一方で、実際の処理内容（例: DynamoDB への書き込み）は `save_item()` のような ハンドラーの外部で定義された関数 に切り出されています。

これにより、ビジネスロジックは Lambda の実行環境やイベント形式に依存せず、テストや再利用が容易な構造になっています。

■ Lambda レイヤーを使用して再利用可能なコードを定義する



Lambda レイヤーは、明示的にアプリケーションコードを再利用するために設計されています。関数のランタイムに適合するレイヤーを使用して、再利用可能なモジュールやライブラリをアタッチできます。

関数に関連付けられるレイヤーの数は限られていますが、レイヤーは任意の関数で使用できます。パブリックでない場合は、レイヤーの表示や使用にはアクセス許可が必要となります。レイヤーを単独で実行することはできません。

レイヤーを使用すると、実際の Lambda 関数のデプロイサイズが小さくなり、CI/CD サイクルの速度が上がります。例えば、Lambda 関数のサイズを 10 MB から 2 MB に縮小できる可能性があります。1 つのレイヤーに複数のランタイムを指定できます。ランタイムは、その中にあるコードの種類を検査するものではなく、タグであることに注意してください。例えば、ランタイムで Node.js と指定しても、Python コードが含まれていれば、そのレイヤーは Python 関数で使えるようになります。ただし、関数が実行しようとするとき、そのレイヤーはエラーとなります。

■ 環境変数を使用して運用パラメータを関数に渡す

関数コード

- ランタイム
- ハンドラーメソッド
- アプリケーションコード
- 環境変数



関数設定

```
import json
import os
import boto3

dynamodb = boto3.client("dynamodb")
table_name = os.environ["SAMPLE_TABLE"]

# -----
# ビジネスロジック
# -----
def save_item(item):
    dynamodb.put_item(
        TableName=table_name,
        Item=item
    )
```

環境変数を使用して、シークレットを安全に保存し、コードを更新することなく関数の動作を調整できます。環境変数は、関数のバージョン固有の設定に保存される文字列のペアです。ランタイムの基盤となる呼び出し環境には、関数コードからアクセスできる追加のライブラリと環境変数が用意されています。独自の環境変数を作成して、コードで使用することもできます。

バージョンを公開するとき、他のバージョン固有の設定と共に、そのバージョンの環境変数がロックされます。

■ 冪等性の重要性

- 冪等性: 同じ入力を複数回処理しても結果に影響がない
- 冪等関数は次の要素からなる
 - 少なくとも 1 回配信
 - 分散型ロジック
 - 再試行動作

コードが冪等である場合は、同じ入力を複数回処理しても、その入力で最初に実行された後の結果には影響しません。

サーバーレスアプリケーションでこれが特に重要な理由はいくつかあります。

1. 多くのイベントソースは、少なくとも 1 回の配信を行います。これは、レコードが複数回配信される可能性があることを意味します。例えば、Amazon SNS と Amazon SQS の両方が 1 回以上配信されます。Lambda が内部キューを使用して非同期リクエストを処理する場合、関数は同じリクエストを 2 回取得できます。
2. 分散ロジックと分散データストアを使用すると、関数の 1 つが同じリクエストを複数回受信するシナリオが発生する可能性が高くなります。
3. レコードのバッチ (Amazon Kinesis Data Streams や Amazon SQS イベントソースなど) を処理する際の再試行の動作には、同じレコードを Lambda 関数に複数回送信することが含まれています。

関数コードは重複レコードを認識して、最初に処理するだけです。例えば、受信イベントの一意の値をチェックし、データベースに既に存在する場合は無視するコードをハンドラーに追加します。

■ イベントソースの一意の識別子を使用して冪等性をチェックする

```
"Records": [
  {
    "messageId": "059f36b4-87a3-44ab-83d2-661975830a7d",
    "receiptHandle": "AQEBWJnKyrHigUMZj6rYigCgxlAs3SLy0a...",
    "body": "Test message.",
    "attributes": {
      "ApproximateReceiveCount": "1",
      "SentTimestamp": "1545082649183",
      "SenderId": "AIDAIENQZJOL023YVJ4VO",
      "ApproximateFirstReceiveTimestamp": "1545082649185"
    },
    "messageAttributes": {},
    "md5OfBody": "e4e68fb7bd0e697a0ae8f1bb342846b3",
    "eventSource": "aws:sqs",
    "eventSourceARN": "arn:aws:sqs:us-east-2:123456789012:my-queue",
    "awsRegion": "us-east-2"
  },
],
```

SQS キュー上の
メッセージごとに一意

イベントペイロード
ごとに一意

冪等関数は次の要素からなります。

- 少なくとも 1 回の配信
- 分散ロジック
- 再試行動作

すべてのイベントソースには一意の ID がいくつかあります。

例えば SQS 標準キューイベントの例があります (スライドで抜粋)。

MessageID は、Amazon SQS が生成する一意の識別子です。SQS キュー上のレコードの一意性を検証する場合は、messageID 属性が利用できます。

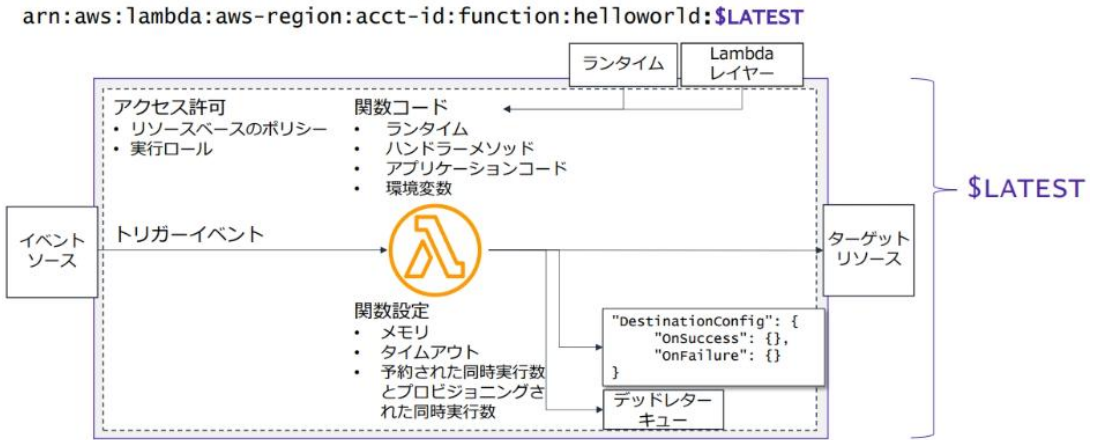
また、実際にペイロードの一意性を検証する必要がある場合もあります。例えば、アップストリームのプロデューサーが同じレコードを Amazon SQS に 2 回送信した場合、それぞれのイベントには Amazon SQS で一意の messageId がありますが、実際はアプリケーションから見ると同じペイロードになります。Amazon SQS イベントで md5OfBody 属性を使用すると、同じペイロードを 2 回処理しないようにすることができます。このフィールドは、ペイロードの一意のハッシュを表します。

先入れ先出し (FIFO) キューには、イベントにおいて、重複を排除し、レコードの順序を保ちながら 1 回限りの配信を行う優先順位付けに関する追加属性があります。同じペイロードを 2 回処理することを防止するコードを入れる必要もあります。

同様に、FIFO SNS トピックとすべての Kinesis データストリームは、重複レコードが処理されないように設計されていますが、Lambda 関数が同一レコードを受信するような状況になるシナリオは存在します。イベントの一意性を常にチェックしてください。

■ バージョンとエイリアス

■ 関数バージョンは、トリガー、アクセス許可、設定、コードを含む



Lambda 関数のバージョンは、ここで説明したすべての要素で構成されます。

- 関数が呼び出されるタイミングを定義するトリガー
- 関数を呼び出すことができるユーザーとその関数で操作できるものを定義するアクセス許可
- Lambda の呼び出し環境の管理方法に影響するように選択した構成
- 実際の関数コード

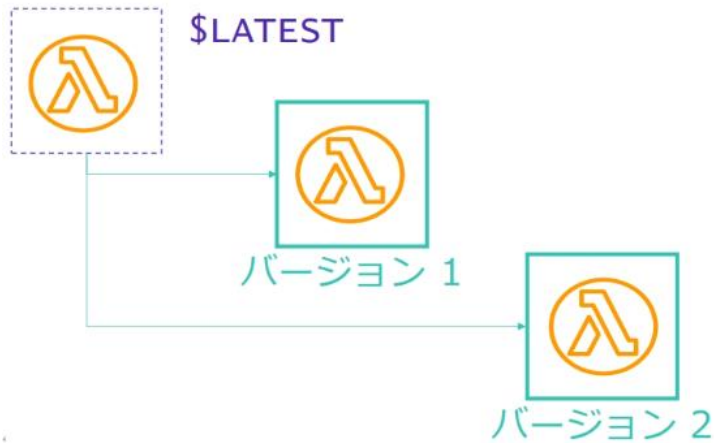
関数を作成するときの唯一のバージョンは \$LATEST です。

関数には、次の 2 つの Amazon リソースネーム (ARN) が関連付けられています。

1. 修飾 ARN - バージョンのサフィックスが付いた関数 ARN です。
2. 非修飾 ARN - バージョンのサフィックスが付いていない関数 ARN です。

■ パブリッシュするとイミュータブルバージョンが作成される

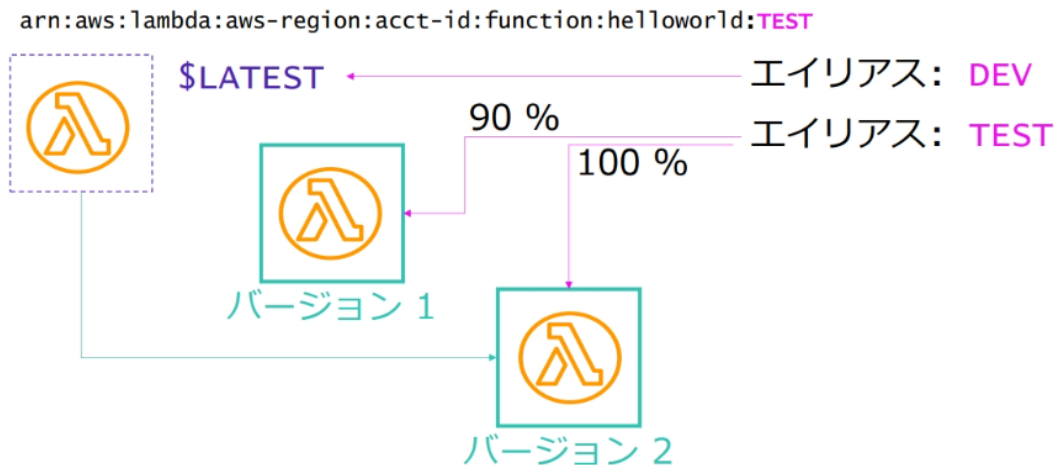
arn:aws:lambda:aws-region:acct-id:function:helloworld:2



Lambda 関数の更新を保存すると、`$LATEST` がデフォルトで更新されます。関数のバージョンを「公開」するオプションを選択すると、Lambda によって未公開の `$LATEST` バージョンのコピーが作成され、連番が付与されます。そのバージョンはイミュータブルです。コードとほとんどの設定は、そのバージョンのユーザー向けに一貫したエクスペリエンスを提供するためにロックされます。そのバージョンの修飾 ARN は、バージョン番号を修飾子として付加します。`$LATEST` コードを編集し、必要に応じて新しいバージョンを公開します。

エイリアスは、関数のバージョンを管理する強力な方法です。

■ エイリアスでバージョンの指定と 2 つのバージョン間でのトラフィックのシフトが可能となる



48

エイリアスを作成して \$LATEST を含む任意のバージョンの関数と関連付けることができます。これにより、エイリアスを ARN の修飾子として使用して関数を参照できます。したがって、関数を参照する必要がある場合は、どこでもエイリアスを使用できます。関数コードが更新されると、その関数を参照する他のリソースのコードを変更しなくても、新しいバージョンのコードでエイリアスをポイントできます。

エイリアスでよく使用するのは、DEV エイリアスと TEST エイリアスです。バージョン 2 のユニットテストが完了し、より広範なテストを行う準備ができると、バージョン 1 ではなくバージョン 2 で TEST エイリアスをポイントできます。

Lambda では、エイリアスの転送機能を使用して 2 つのバージョンをポイントするエイリアスを設定し、トラフィックを一定の割合で各バージョンに送信することもできます。例えば、バージョン 2 をテストバージョンとして初めて使用する場合、統合テストで予期しない問題が起きないことを確認するために、トラフィックのごく一部だけを送信できます。その後、期待どおりに動作していると確信できると、トラフィックの 100% をバージョン 2 に送信するように設定を変更できます。

■ 関数の実行とテスト

■ コンソールでの関数のテスト

■ コンソールでのテスト:

- 複数のテストイベントを設定する
- コンソールまたは CloudWatch に結果を表示する
- Lambda Blueprint には設定済みのテストイベントが用意されている

テストイベントを作成してから、コンソールから任意のバージョンの関数をテストできます。結果は、メインコンソールセクションの上にあるパネルに表示されます。

Lambda にはメトリクスとログが組み込まれており、Lambda コンソールから直接アクセスすることが可能で、CloudWatch へのリンクを介して利用することもできます。

■ Lambda エラー処理

一般的な 2 種類のエラー



デベロッパーは、関数に関して呼び出しエラーと関数エラーの両方に注意してください。呼び出しエラーは、Lambda サービスが関数を呼び出すことができないときに発生します。呼び出しエラーが発生するシナリオは 2 つあります。

- 最初のシナリオは、イベントソースによって Lambda に渡されたイベントに問題がある場合に発生します。関数を呼び出すための適切なアクセス許可がソースにない、JSON が無効である、ペイロードが大きすぎる、などです。
- 2 番目のシナリオは、Lambda サービスがイベントを受信しても、関数の呼び出しを開始できない場合に発生します。このタイプのエラーの最も一般的な理由は、新しい呼び出し環境を開始するのに十分な同時実行数がないことで、そのため Lambda はリクエストを制限します。

呼び出しエラーによって、エラーコード400番台 または 500番台が発生します。

軽食レストランに例えると、最初のエラータイプは、料理人が注文書の筆跡が読み取れないために、その注文に関する作業を開始できない場合に発生することがあります。2 番目の呼び出しエラータイプは、「料理人が注文を受け取っても、ガスレンジがないために調理を開始できない」という以前に聞いた例です。

もう 1 つのタイプのエラーは関数エラーです。この場合、Lambda は関数を正常に呼び出しますが、何らかの問題（関数のタイムアウト、例外のスローなど）が発生します。関数エラーが発生すると、Lambda は 400 または 500 のエラーは生成しません。Lambda は、X-Amz-Function-Error という名前のヘッダーに、エラーメッセージなどの詳細情報を記述した JSON 形式の応答を付けて、関数エラーを示します。

軽食レストランに例えると、これはパンケーキを調理しているときに、その中の 1 つを床にひっくり返すようなものかもしれません。

Lambda などのマネージドサーバーレスサービスには、ここで使用できるエラー処理メカニズムが組み込まれています。デベロッパーは、これらのメカニズムの内容、Lambda イベントソース間でのメカニズムの違い、問題が発生したときの調査項目を理解する必要があります。

同期イベントソースはすべてのエラーを処理する必要がある

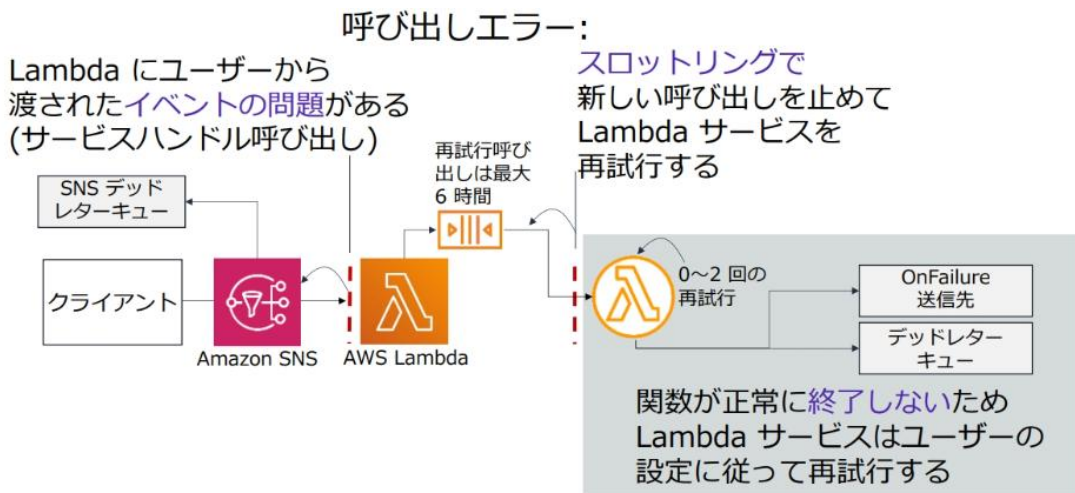


同期イベントソースでは、リクエスト/レスポンス API 呼び出しであることを覚えておいてください。イベントソースまたは接続対象のクライアントは、関数の呼び出し中または実行中に発生したエラーを処理する必要があります。例えば、API Gateway を使用して Lambda 関数へのクライアント要求をプロキシする場合、そのクライアントは戻ってくるエラーを処理する必要があります。

AWS CLI と AWS SDK には、デフォルトでバックオフと再試行が含まれているため、これらを活用してエラーに対処します。

軽食レストランに例えると、同期トランザクションはカウンターでパイを一切れ注文しています。カウンターの向こうの人は、ケースからパイを取り出してお客様に渡し、トランザクションを完了する必要があります。カウンターの人は何か問題がある場合（パイを落とした、チェリーパイが残っていないことが分かった、など）、トランザクションを完了して、お客様にパイがないというメッセージを返す必要があります。

■ 非同期イベントソースは Lambda とエラー処理を共有する



非同期呼び出しでは、呼び出しをリクエストするイベントソースは関数の結果は待ちませんが、リクエストが正常に伝わったことを確認する責任があります。

軽食レストランの例えでは、注文をキッチンマネージャーに渡しているウェ이터です。ウェ이터は、注文が正常に完了したことを確認するまではカウンターで待ちませんが、料理人が注文を受け取る方法と場所で確実に注文を受け渡す必要があります。キッチンマネージャーが注文を読み取ることができない場合やお客様がメニューにない料理をリクエストした場合は、ウェ이터はそのエラーに対処する必要があります。キッチンマネージャーが休憩で外出中であれば、ウェ이터はそれに対処する必要があります。

非同期イベントソースの場合、クライアントまたは呼び出しサービスには、アクセス許可の問題や無効な JSON など、Lambda の関数呼び出しを妨げるエラーのほとんどに対応する責任があります。独自の非同期イベントソースを作成する場合、このようなタイプのエラーを管理し、再試行のメカニズムを設計する必要があります。

AWS のサービスを非同期イベントソースとして使用すると、これらのタイプの先行呼び出しエラーは、関数を呼び出すサービスに関連付けられた再試行動作に基づいて再試行または破棄されます。例えば、Amazon SNS がイベントソースである場合は、リクエストを Lambda に渡せなければ、独自の再試行ポリシーが適用されます。

Amazon SNS では、クライアント側のエラーが原因で発生した配信は再試行されません。例えば、削除された関数にメッセージを配信しようとしています。(チェリーパイがないのに、注文しようとしてしました。もう一度注文しようとしても意味がありません)。

サーバー側のエラーは、サブスクライブされたエンドポイントを担うシステムが利用できなくなったり、Amazon SNS からの有効なリクエストを処理できないことを示す例外を返す場合に発生します(注文は有効ですが、料理人は休憩中です)。Amazon SNS では、エンドポイントのタイプの再試行ポリシーに基づいて、サーバー側のエラーが再試行されます。Lambda または別の AWS マネージドエンドポイントがサブスクライバーである場合、再試行の期間が非常に長く延長され、試行間の間隔が長くなります。

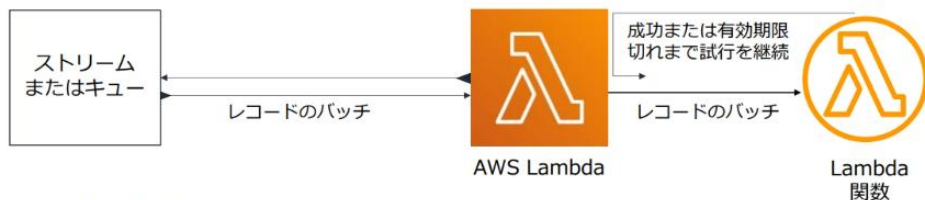
Amazon SNS がクライアント側のエラーを受信した場合、またはサーバー側のエラーの再試行が終わった場合は、その SNS トピックに関連付けられたデッドレターキューが設定されていなければ、メッセージは破棄されます。

非同期イベントソースがリクエストを Lambda に正常に渡すと、イベントソースは成功したことを示す 202 メッセージを受信します。この時点で、Lambda は独自の内部キューからイベントを管理しています。それ以降に発生する呼び出しエラー（スロットリングエラーなど）は Lambda の責任であり、Lambda は呼び出しを再試行します（キッチンがバックアップされている場合でも、キッチンマネージャは有効な注文を処理します）。

Lambda では、このタイプの障害の再試行の頻度とバックオフパターンが制御されますが、関数の設定でエラーとなった呼び出しを再試行する時間を設定できます（最大 6 時間）。Lambda によって関数が正常に起動されても、関数で例外がスローされるか、完了されなかった場合、Lambda によってさらに 2 回まで関数の実行が再試行されます。この再試行の値は、関数設定で 0~2 に設定できます。

エラーが設定した時間を超過して継続する Lambda 呼び出しエラー、および再試行設定を使い切った関数エラーは OnFailure 送信先またはデッドレターキューに送信できます。アプリケーションには、このようなレコードを処理するためのメソッドを組み込む必要があります。例えば、何らかの条件が解決された後にレコードを再生したり、解決のためにそのレコードを別のプロセスに送信したりします。EventBridge がイベントソースの場合は、イベントをアーカイブして後で再生されるようにイベント再生機能を設定できます。これは、処理エラーから回復する場合に便利です。

■ Lambda はポーリングイベントソースのエラーを管理する



エラー処理の設定

- **キュー**: コード内の部分的な障害を処理する。ソースキューのリドライブポリシーとデッドレターキューを設定する
- **ストリーム**: エラーまたはチェックポイント時の最大再試行回数とバッチ分割を設定し、OnFailure 送信先を設定する

ストリームとキューの両方を使用すると、レコードはアップストリームで生成され、ストリームまたはキューに追加されます。Lambda は、キューまたはストリームからレコードのバッチを取得し、そのバッチと同期して関数を呼び出します。

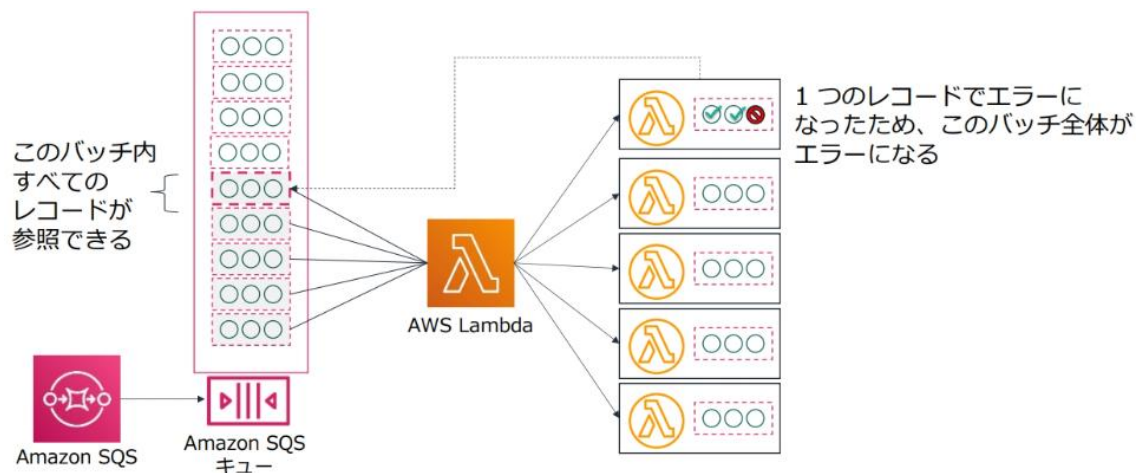
一般的に、Lambda はそれ自身とイベントソース間のエラー処理を受け持っており、Lambda は成功する（デフォルト）かレコードの有効期限が切れるまでバッチの試行を継続します。これらのイベントソースについては、Lambda でエラーの数が増える可能性があることを示すキューとストリームの動作に特に注意してください。

キューとストリームの両方で、デフォルトのエラー処理動作を変更して、エラーとなったレコードが邪魔にならないようにする設定オプションがいくつかあり、これらを使用することをお勧めします。

また、Lambda はバッチでレコードを処理するため、バッチ内のレコードの一部が成功し、他のレコードがエラーになる可能性があります。この方法では、レコードが複数回処理される可能性も高くなります。デベロッパーは、部分的なエラーの説明責任を負う必要があり、重複レコードの説明責任のベストプラクティスに確実に従う必要があることを意味します。

■ SQS キューのエラー処理

■ Lambda では成功したバッチは削除されるが、レコードのどれか 1 つがエラーになると、そのバッチはエラーとなる



SQS キューがイベントソースとして使用されると、Lambda によってバッチのキューがポーリングされ、レコードのバッチごとに 1 つの呼び出しが同期的に開始されます。Lambda では、5 つのポーリングプロセスから開始します。このポーリングプロセスによって、関数の 5 つの同時実行インスタンスが呼び出されます。

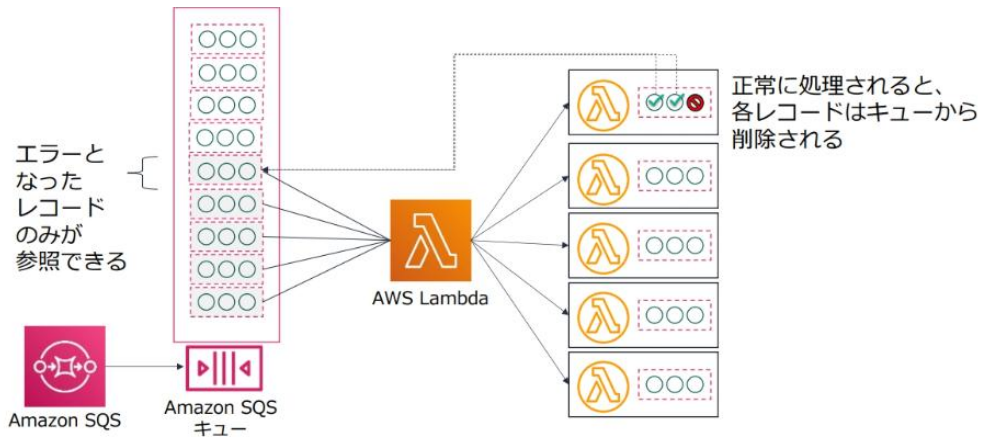
バッチ内のレコードがエラーになると、Lambda サービスではこれをエラーとして識別し、バッチ全体がエラーとなります。このエラーには、バッチ内のすべてのレコードが処理される前の関数のタイムアウトも含まれます。バッチがエラーになると、そのレコードは再びキューで参照できるようになります。

これが日常的に発生する場合は、バッチの処理にかかる平均時間に対して、指定した Lambda 関数のタイムアウトが短すぎる可能性があります。例えば、関数のタイムアウトが 30 秒に設定されていて、3 つのレコードのバッチの処理がそれぞれ平均 10~15 秒かかる場合は、関数はすべてのレコードが処理される前に頻繁にタイムアウトします。

追加のエラー処理オプションを構成しなければ、これらのレコードは、キューに設定されたメッセージ保持期間を超過するまで再試行されます。

これは、幂等性が重要な理由の 1 つです。レコードは、正常に完了しないバッチの一部として繰り返し処理されることがあります。

■ SQS キューで部分的な障害を処理するためのベストプラクティス



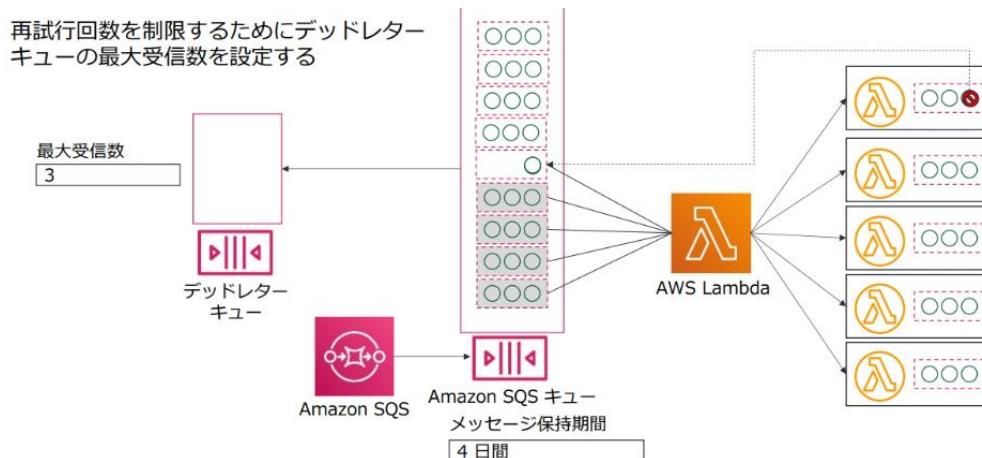
バッチの部分的なエラーのために Lambda で同じレコードが何度も処理し続けられないように、正常に処理されたレコードは処理が完了したらすぐに Lambda 関数コードでキューから削除することをお勧めします。こうすると、バッチの一部がエラーになった場合、エラーになったレコードだけがキューに再び参照できるようになります。

新機能

AWS Lambdaは、イベントソースとしてSQSの部分的なバッチレスポンスをサポートするようになりました。この機能を使用すると、SQSキューのメッセージの処理に失敗した場合、Lambdaはメッセージキュー内のレコードのバッチを部分的に成功したものとしてマークし、失敗したレコードのみの再処理を許可します。

AWS Lambdaは、バッチレベルではなくレコードレベルで情報を処理することにより、繰り返しのデータ転送の必要性を排除し、スループットを向上させ、AmazonSQSメッセージキュー処理をより効率的にします。

■ キューでのエラー処理の設定



Lambda によって SQS キューが管理されていますが、長いエラーレコードの再試行方法を設定するためのオプションがあります。

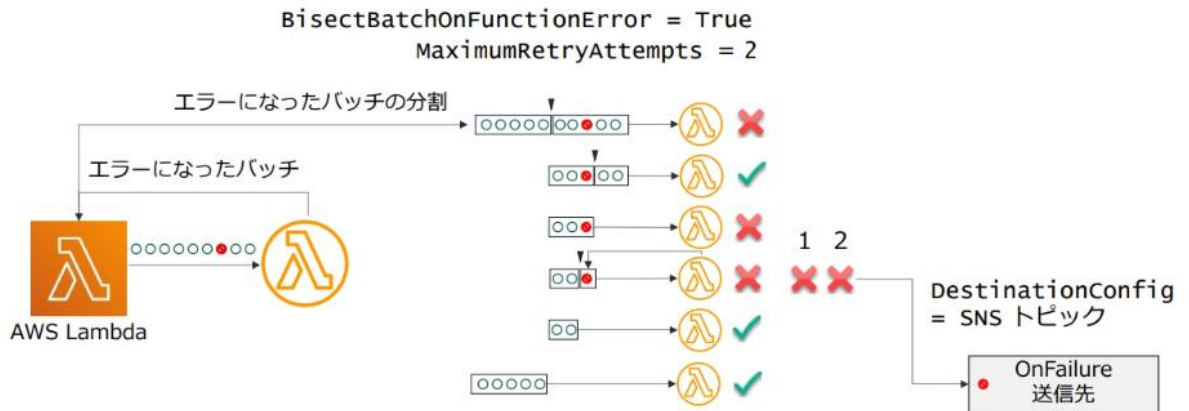
キュー自体の保持期間を制限できます。設定した制限時間を超えてキューに残っている場合、メッセージは破棄されます。

「最大受信数」の値と合わせてデッドレターキューを設定するのがベストプラクティスです。これにより、Lambda でエラーとなったレコードが再試行される回数が制限されます。レコードが最大受信数の値に達すると、そのレコードはデッドレターキューに送信されます。このキューでは、エラーとなったレコードに対してオフライン分析または別の自動処理を実行できます。

注: Amazon SQS がイベントソースの場合、設定したデッドレターキューは Lambda 関数ではなく、ソースキューにあります。関数のデッドレターキューは、関数の非同期呼び出しに使用されます。

■ ストリームのエラー処理

関数エラー時の分割とストリームでの最大再試行回数を使用する



ストリーム処理が無期限にブロックされないように、最大再試行回数または最大経過時間を使用してエラー時の分割を行うことをお勧めします。

関数エラー時のバッチ分割は、エラーとなったバッチを2つに分割して、各バッチを別々に再試行するように Lambda に指示します。

最大再試行回数および最大レコード経過時間を使用すると、エラーとなったバッチの再試行回数または継続時間を制限できます。

OnFailure 送信先を使用すると、エラーとなったレコードを SNS トピックまたは SQS キューに送信し、関数にロジックを追加しなくても、オフラインで処理できます。

このスライドでは、10 レコードのバッチでこれがどのような動きをするかを示しています。

この例では、`BisectOnFunctionError = True`、`MaximumRetryAttempts = 2`、`DestinationConfig` には SNS トピックをポイントしている `OnFailure` 送信先が含まれます。この例では、3 番目のレコード (このバッチの 10) は関数エラーを返します。

関数がエラーを返すと、Lambda はバッチを2つに分割し、それらを個別に関数に送信してレコードの順序を維持します。また、Lambda はバッチを分割するたびに、再試行値と最大期間値をリセットします。これで5レコードのバッチ2つになりました。Lambda は最初の5レコードのバッチを送信してエラーになるので、分割プロセスが繰り返されます。Lambda はエラーになったバッチを分割して、レコードが2つのバッチと3つのバッチを生成します。再試行値と最大期間値をリセットし、これらの2つのバッチのうち最初のバッチを処理用に送信します。

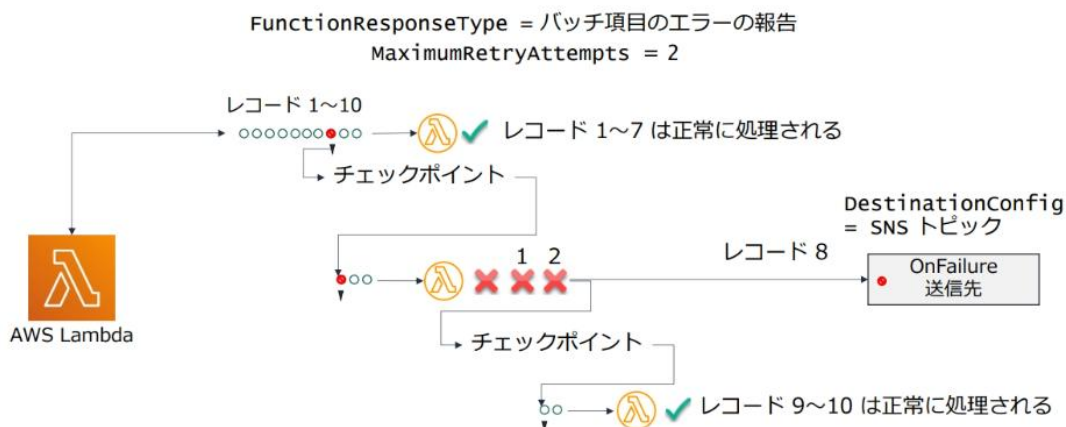
今回は、2つのレコードのバッチが正常に処理されるため、Lambda は3つのレコードのバッチを関数に送信します。そのバッチはエラーとなり、Lambda はそのバッチを分割して、1つのレコード（不良レコード）のバッチと2つのレコードのバッチになります。

Lambda は不良レコードでバッチを送信してエラーになりますが、分割するものは何も残っていません。これで最大再試行回数と最大有効期間の設定が有効になります。この例では、関数はレコードを2回再試行し、エラーが続くと、OnFailure 送信先に設定された SNS トピックにレコードを送信します。エラーが途中で繰り返されるため、Lambda は作成された小さなバッチのそれぞれを元に戻し、常にレコードの順序を維持します。つまり Lambda は2の未処理のバッチを処理し、次に5の未処理のバッチを処理します。

その時点で、元の10のバッチが成功としてマークされ、Lambda はストリーム上のポインタを次のレコードのバッチの先頭に移動します。

この方法の欠点の1つは、レコードが複数回処理される場合があることです。

■ チェックポイントを使用してエラーを処理し、順序を維持する



前のスライドで示したように、BisectBatchOnError を使用すると、バッチが分割および再試行されるときに、重複処理が発生する可能性があります。

FunctionResponseType を Report Batch Item Failure に等しいと設定した場合、バッチの処理がエラーになると、最後に成功したメッセージの後のレコードのみが再試行されます。この設定により、処理の重複を減らし、処理のエラーに対応するための選択肢を広げることができます。処理のエラーが生じた場合、このチェックポイント機能が有効化されていれば、Lambda はこの機能を他のメカニズムより優先的に処理し、処理の重複を最小化します。

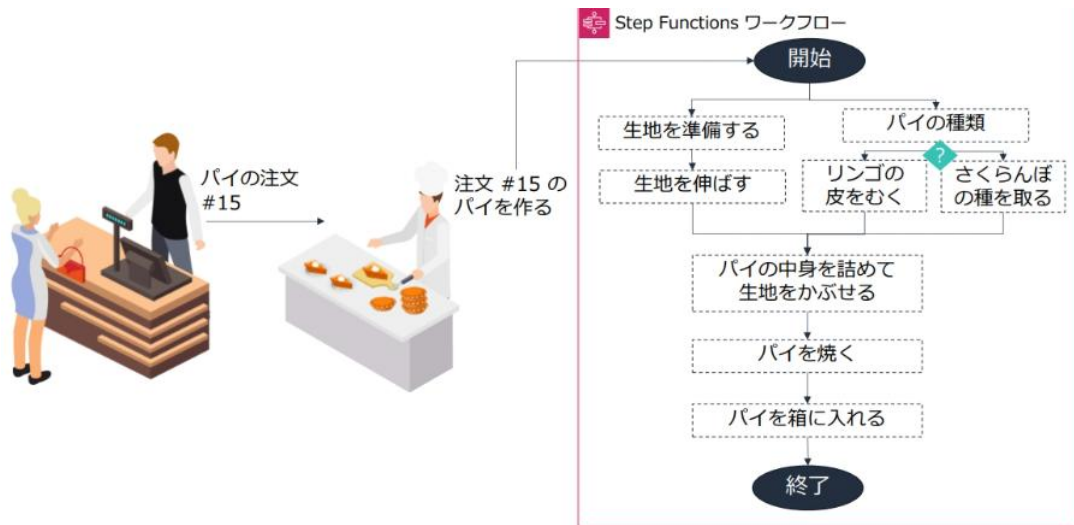
この例では、10 レコードの処理のバッチの最初の 7 レコードは正常に処理されますが、8 番目のレコードはエラーになります。チェックポイントを有効にすると、10 レコードのバッチ全体を再試行したり、バッチを 2 つに分割するのではなく、Lambda はチェックポイントから再試行を開始します。したがって、この例では、Lambda はレコード 8 で再試行を開始します。チェックポイントを使用するには、Lambda 関数がエラーになったシーケンス識別子を返す必要があります。

MaximumRetryAttempts には、再試行を制限するオプションが用意されており、エラーが続くレコードを OnFailure 送信先に送信できます。この例では、レコード 8 は引き続きエラーになるため、2 回の再試行後、レコードが OnFailure 送信先に送信されるまで、チェックポイントは同じポイントに留まります。チェックポイントでは、レコード 9 がバッチ内の次の未処理レコードであることが検出され、そこから処理が開始されるようになります。レコード 9 と 10 は正常に処理され、レコード 1~7 と 9~10 の処理順序が保持されます。

08 : Step Functionsを使用した オーケストレーション

Chapter 8

1つのリクエストから一連のステップを開始する



パン屋の例に戻しましょう。パイを作る必要がある注文は、相互に関連しているもののそれぞれが個別のステップで構成されます。一部のステップは、最初または順番に完了する必要があり、一部は並行して作業することができます。他のステップより時間がかかるステップもあります。専門知識を持つ各ステップの担当者がそのステップを実行します。

各チームメンバーから次のチームメンバーに引き渡すことで、各チームメンバーが前後の状況と問題があった場合の対処方法を把握できるようにします。つまり、各スタッフがある程度の知識を必要とし、専門分野外のことに対して努力を費やすということです。小規模の作業の場合はこのアプローチで対応できますが、計画しているのは大量のパイを作ることです。

もう1つの潜在的な問題は、従業員に対して時間給を支払っているため、長時間かかるタスクが完了するのを待ってアイドル状態にしたいということです。例えば、パイを焼く担当者が、パイが焼けるまでオープンな窓から中をのぞき込んで立っているようなことは望ましいとは言えません。パイが焼けたら担当者が戻って、パイを取り出すようにしたいと考えるはずですが。

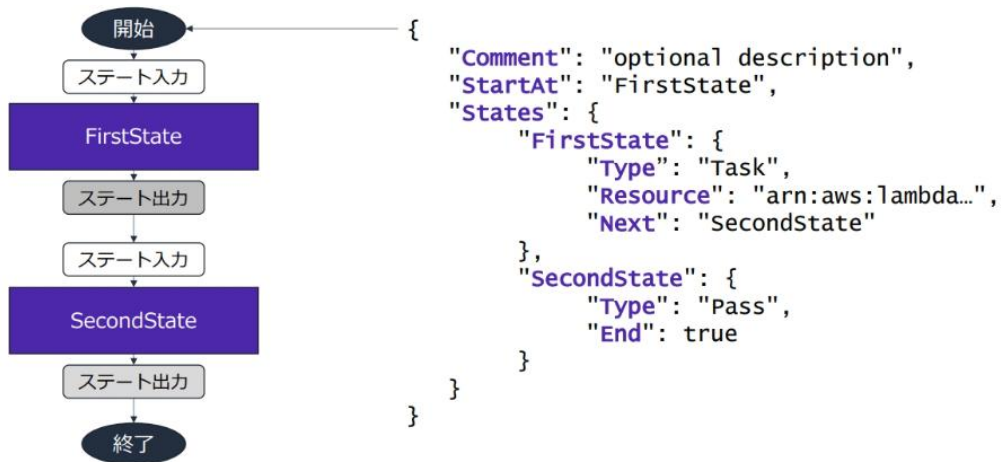
物事をスムーズに進め、専門家がそれぞれの専門分野で作業できるようにするために、ステップのフローを管理し、他の人にステータスを知らせる方法を追加することができます。

■ 開発者向け機能

- ワークフローをステートマシンとして定義する
- ステートメント言語を使用してオーケストレーションロジックを処理し、コード外で状態を維持する
- コンソールで利用可能なテンプレートとコードスニペットから開始する
- 自動生成されたフロー図と実行履歴を使用して進行状況を監視する

AWS Step Functions コンソールでは、テンプレートやスニペットにアクセスして、そこから開始することができます。

■ ステートが入力を取得し、何かを実行し、出力を渡す



ステートは、ステートマシンで関数を実行する要素です。ステート名は、識別に使用する文字列です。

Step Functions の実行は JSON テキストを入力として受け取り、その入力をワークフローの最初のステートに渡します。

ステートマシンには常に StartAt フィールドがあり、このフィールドは処理が開始されるステートを名前で示します。ステートマシンには常に States セクションがあり、このセクションによってステートマシン内のすべてのステートが定義されます。

各ステートには、ステートの種類を示す Type フィールドが必須です。各ステートタイプは、異なるタイプのロジック用に設計されており、一部のフィールドは特定のステートタイプに関連しています。

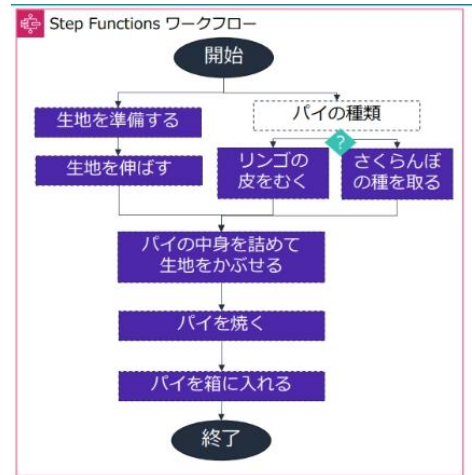
ほとんどのステートに、next または end フィールドがあり、そのステートの完了時に何をすべきかをステートマシンに指示します。ステートはリストされた順序では処理されません。next フィールドによって、オペレーションの順序が決定します。

この例では、FirstState は AWS Lambda 関数を実行する Task ステートであり、次のステップが SecondState であることを示します。secondState は Pass ステートタイプであり、ステート完了後に処理が終了することを示します。

Task ステートによって処理が実行される

■ Task ステートにより、次の処理が実行される

- Lambda 関数
- 他のサービスの API アクションへのパラメータの受け渡し
- アクティビティ（アクティビティワーカー経由でホストおよび実行するアプリケーション）



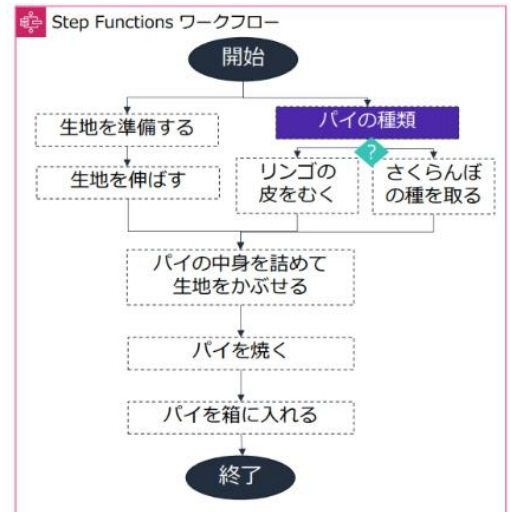
パイの例では、記載されている項目のほとんどが Task ステートであり、各ステップで処理が行われます。これらのステップでは、個々の Lambda 関数を表すことができますが、別の場所で記述してホストした別のサービスまたはアプリケーションコードとの相互作用を表すこともできます。

サポートされているサービス統合:

- 最適化された統合 (17 サービス)
- AWS SDK 統合 (200以上のサービス)

Choice ステートは分岐ロジックを提供する

- Choice ステートなどの状態はタスクをオーケストレーションするが、処理自体は実行しない
- Choice ルールによって、ステートマシンが次に何をすることが決定される

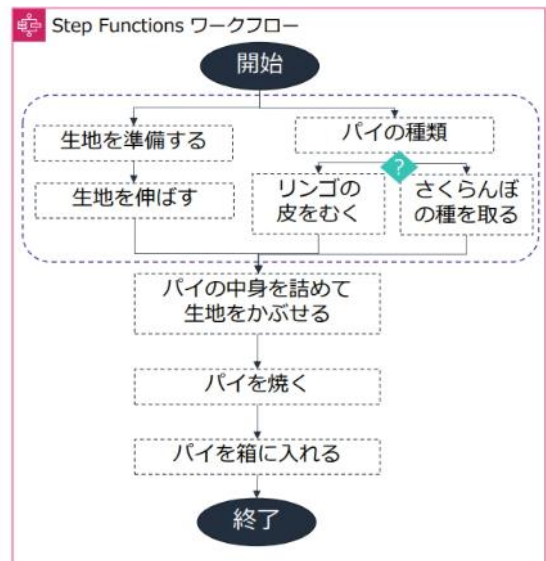


この例では、Choice ステートを「パイのタイプ」としています。ルールでは、次のステップとしてりんごの皮を剥くか、さくらんぼの種を取るかの、いずれかのフローに進むように指示しています。また、データがルールのいずれとも一致しない場合にパスが存在するように、デフォルトのオプションを指定することもできます。

Parallel ステートは並列ブランチを開始する

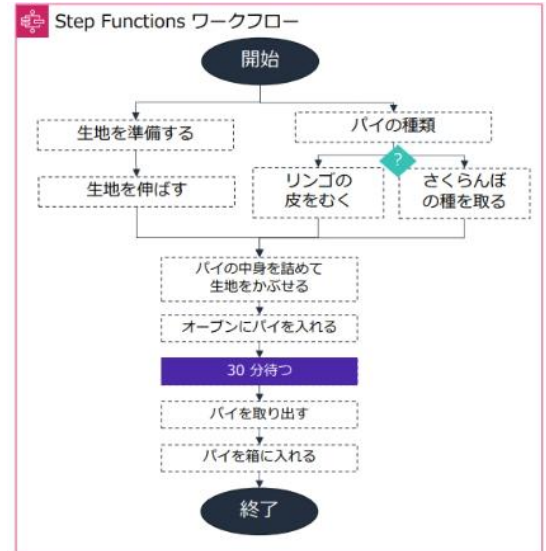
■ Step Functions:

- ブランチをできる限り同時に開始する
- すべてのブランチが終了するまで待つ



Wait ステートによる一時停止処理

- Wait ステートによって、特定の時間枠または相対時間枠の処理を一時停止することができる
- Lambda 関数で処理の完了を待機させる代わりに、Wait ステートを使用する



この例では、「パイを焼く」関数を「オープンにパイを入れる」関数に置き換え、「パイを取り出す」関数を 30 分間の Wait ステートに設定します。

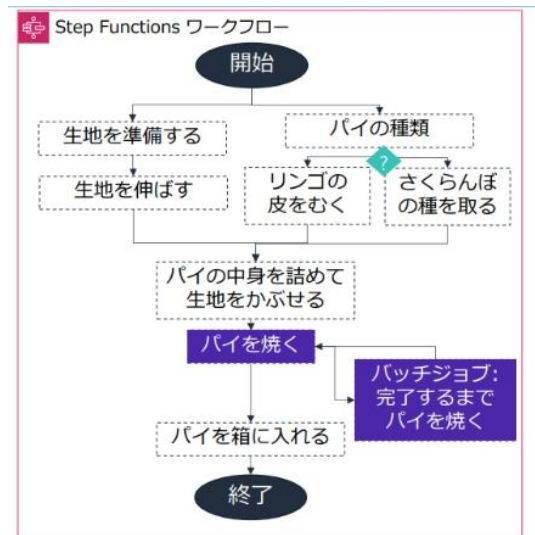
この方が Lambda にとって効率的で低コストです。Lambda 関数に待機時間を設定すると、そのアイドル時間の料金を支払うことになります。Step Functions のコストは、ある状態から別の状態への遷移にかかる時間ではなく、状態遷移の数に基づきます。

多くの場合、待ち時間は外的要因に依存します。Task ステートでいくつかのオプションを使用して、不特定の時間にわたって待つことができます。

- ジョブが完了するまで待つ
- タスクトークンのコールバックまで待機する

Task ステートでジョブが完了するのを待つ

- AWS リソースで .sync というサフィックスを使用し、外部タスクが完了するまで処理を一時停止する
- このオプションは、一部の AWS のサービス統合で利用可能

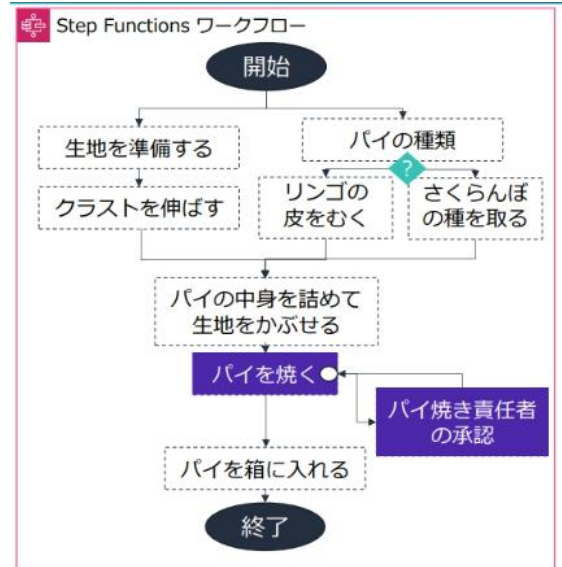


AWS リソース (Lambda 関数、Amazon Simple Notification Service (Amazon SNS)、Task ステートなど) を指定した場合、Step Functions は HTTP レスポンスを待ち、次の状態に進みます。Step Functions は、ジョブが完了するのを待ちません。

AWS Batch や Amazon Elastic Container Service (Amazon ECS) などの一部の AWS のサービスの統合では、.sync というサフィックスを使用して、Step Functions タスクが外部タスクの完了を待つことができる統合パターンをサポートしています。

Task ステートでタスクトークンによるコールバックを待つ

- 外部アクションを待機する必要があるタスクには、コールバックを使用する
- タスクトークンを渡すと、Step Functions はトークンが返されるまで一時停止する



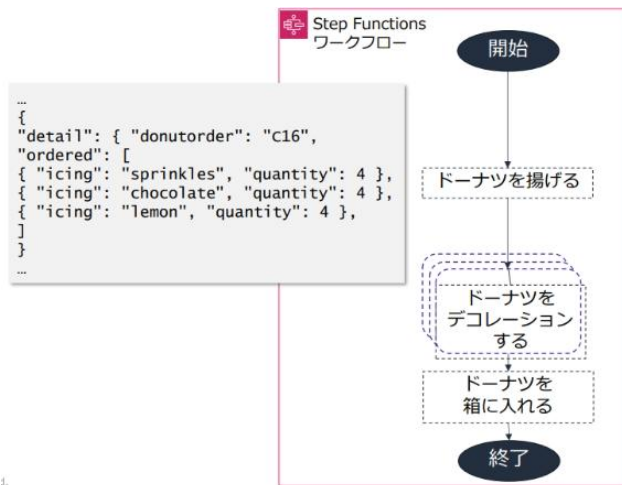
または、外部からの応答（例えば、人的承認や要求を処理して応答する必要があるサードパーティーシステム）を取得するまで、処理を一時停止する必要があります。

Step Functions と統合されている一部の AWS のサービスでは、渡されたタスクトークンを使用することができ、処理が完了すると戻されます。

これを行うには、「.waitForTaskToken」を使用して、タスクの状態を一時停止してタスクトークンを待つように設定します。

■ Map ステートで入力配列を繰り返し実行する

- Map ステートは、入力 JSON 配列の複数のエントリに対して同じ手順を実行する
- ステートは、配列内の各項目に対してイテレーターのセクションを 1 回実行する



多くの場合、ステップへの入力、は、さまざまな数値を持つ項目のリストです。ステート入力配列の複数のエントリに対して同じ手順を実行するには、Map ステートを使用します。

例えば、ドーナツ注文のステートマシンに切り替えた場合、Map ステートを使用して、顧客のドーナツ注文の配列の各入力に対して「ドーナツを冷ます」関数を実行できます。同じ関数が、個々の配列項目の値を使用して、配列内の各項目に対して実行されます。Map ステートは、関数が配列内のすべての項目を反復処理するまで完了しません。

- Amazon ステートメント言語には、一般的なエラー名を付けた組み込みの文字列セットが定義されており、すべて「States.」というプレフィックスで始まる。
- Task ステートと Parallel ステートには、Retry という名前のフィールドを定義できる。このフィールドの値は Retriers というオブジェクトの配列にする必要がある。

どのステートでも、ランタイムエラーが発生する可能性があります。エラーはさまざまな理由で発生します。

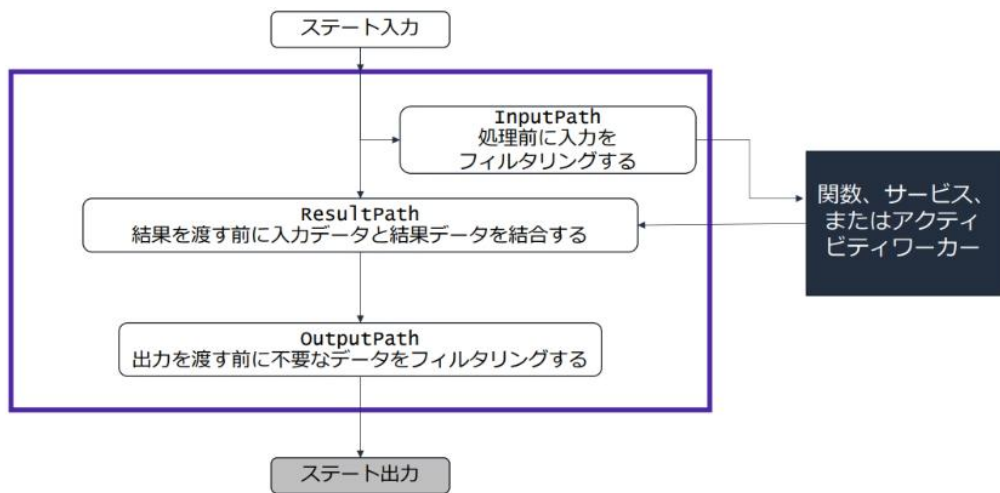
- ステートマシンの定義の問題（例えば、Choice ステートで一致するルールがない）
- タスクの失敗（例えば、Lambda 関数の例外）
- 一時的な問題（例えば、ネットワークパーティションイベント）

デフォルトでは、ステートでエラーが報告されると、Step Functions の実行全体が失敗します。Step Functions は、エラー名と呼ばれる大文字と小文字を区別する文字列を使用して、Amazon ステートメント言語のエラーを識別します。

Amazon ステートメント言語には、一般的なエラー名を付けた組み込みの文字列セットが定義されており、すべて「States」というプレフィックスで始まります。

Task ステートと Parallel ステートには、Retry という名前のフィールドを定義できます。このフィールドの値は Retriers というオブジェクトの配列にする必要があります。個々の Retrier は特定の再試行回数を表します。通常、再試行の間隔は徐々に長くなります。

■ パスによる入力と出力のフィルタリング



各ステート内で、追加のフィールドを使用して入力と出力をフィルタリングすることができます。

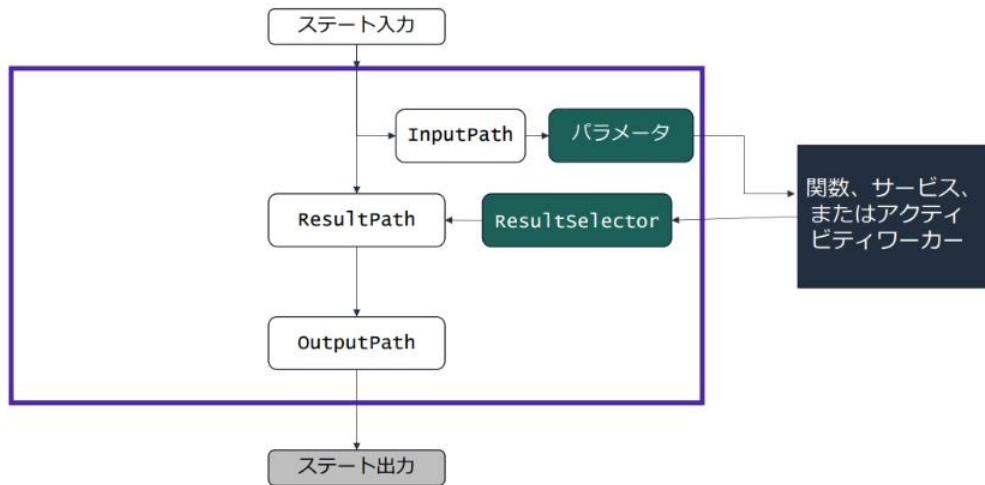
パスを使用すると、JSON の一部を選択できます。Fail ステート以外のすべてのステートでは、InputPath、ResultPath、 および OutputPath を使用して、ステップがデータを処理する前と後にデータをフィルタリングできます。

InputPath を使用すると、Task ステートの関数、サービス、またはワーカーに渡される入力を制限できます。

ResultPath では、結果の受け渡し方法を選択できます。結果は、入力のコピー、生成された結果、またはその両方の組み合わせにすることができます。

OutputPath を使用すると、出力の一部を選択して、次のステートに渡すことができます。

パラメータと ResultSelector による入力と結果の強化



[Parameters] および [ResultSelector] フィールドを使用すると、ステート内の入力データと出力データを強化することができます。

[Parameters] フィールドでは、入力として渡されるキーと値のペアの集合を作成できます。これらには静的な値が有効ですが、入力オブジェクトまたはパスを持つコンテキストオブジェクトから選択することもできます。

コンテキストオブジェクトにはステートマシンに関する情報が含まれおり、個々の実行に固有の情報へアクセスすることができます。実行中、コンテキストオブジェクトにアクセスできる [Parameters] フィールドのデータが入力されます。これは、タスクトークンや Amazon Simple Queue Service (Amazon SQS) メッセージ ID などの場合に便利です。

[ResultSelector] フィールドを使用すると、ResultPath が適用され、Task、Map、Parallel ステートで使用できるようになる前に結果を操作できます。例えば、サービス統合で返されたメタデータの一部を選択し、ResultPath でステートの入力とマージできます。

■ ステートの基本的な操作に組み込み関数を使用する

■ 個別の Task ステートを使用せずに一般的なアクションを実行する

■ 例:

- 文字列を連結する
- 文字列から JSON を作成する
- データから配列を生成する

[Parameters] および [ResultSelector] フィールドは、一部のステートタイプの組み込み関数もサポートしています。組み込み関数を使用すると、文字列の連結、文字列からの JSON の作成、データからの配列生成などの基本的な操作を実行できます。組み込み関数を使用すると、変更されたデータを返す関数を実行する別の Task ステートを作成しなくても、これらのタイプの一般的なアクションを実行できます。

■ サービス間の直接呼び出し

- 他のサービスから AWS Step Functions を呼び出す
- 他のサービスで AWS Step Functions を使用する

■ モニタリングとデプロイ

- AWS X-Ray と AWS Step Functions
- AWS Step Functions と AWS SAM

Step Functions では、他の AWS のサービスとの統合が引き続き追加されており、Step Functions の記述と検証のしやすさが向上しています。

ステートマシンを構築するときは、ワークフローとペイロードに関連する要因に基づいて、呼び出しオプションを確認します。例えば、Amazon API Gateway を直接使用してステートマシンを開始したいのはどういった場合で、API Gateway の背後に Lambda 関数を使用してステートマシンを開始したいのはどういった場合でしょうか。

前述のように、Task ステートから直接多くのサービスと対話することができ、統合パターンにより、ジョブの終了を待機したり、タスクトークンを返すためのロジックをはるかに少ないコーディングで簡単に構築できます。独自のロジックを作成する前に、利用可能なサービスの統合がないか確認するようにしてください。

Step Functions は AWS X-Ray もサポートしており、ステートマシン内で起きていることを追跡することができます。AWS サーバーレスアプリケーションモデル (AWS SAM) 内からコンソールを使用してステートマシンのフローを可視化し、すばやく参照できます。

Standard ワークフローと Express ワークフロー

Standard	Express
最長 1 年間	最長 5 分間
1 回限りのモデル	少なくとも 1 回のモデル
すべての状態遷移で Execution ステートが内部的に保持される	実行の進行状況について、内部的に保持されるステートはない。ロジックはベキ等でなければならない。
非同期タスクによる長時間実行ワークフローに最適 ★	大量のイベント処理ワークロードに最適 ★

例では Standard ワークフローを使用してきましたが、オーケストレーションする内容によっては、Express フローが適している場合があります。

Standard ワークフローは、非同期タスクを使用して長時間実行されるワークフローに最適です。一方 Express ワークフローは、期間が短い大量のイベント処理向けに設計されています。

Express ワークフローは、同期リクエスト/レスポンスのワークフローもサポートしています。そのため、開発者は追加のサービスをポーリングしたり、追加のコードを記述したりすることなく、ワークフローの応答をすばやく受け取ることができます。これは、大量のマイクロサービスのオーケストレーションや HTTPS 経由で通信する高速コンピューティングタスクに有用です。例えば、複数ステップのサインアッププロセスを同期的に処理することができます。

本番ワークフローが Express ワークフローに最も適している場合でも、開発者にとっては、Standard ワークフローを使用してコードまたはワークフローを正しく取得する処理が有用です。これは、エラーが視覚的に表示され、各実行または実行の各ステップの詳細にアクセスしやすい実行履歴が提供されるためです。ワークフローを必要に応じて実行した後に、ワークロードが Express ワークフローに適していることが分かった場合は、Standard ワークフローを Express ワークフローにコピーできます。

【演習3 AWS Step Functions】

タスク 1 : DynamoDB テーブル作成

- ①AWS マネジメントコンソール で DynamoDB を開き、「テーブルを作成」を選択します。
- ②以下、設定でテーブルを作成します。
テーブル名：Orders（注文管理用）
パーティションキー：orderId（String）
キャパシティモード：オンデマンド
- ③もう1つテーブル作成します。
テーブル名：OrderAuditLogs（ログ用）
パーティションキー：eventId（String）
ソートキー：createdAt（String）
キャパシティモード：オンデマンド

タスク 2 : SNS トピック作成

- ①Amazon SNS コンソール を開き、左メニューで「トピック」を選択します。
- ②「トピックを作成」をクリックします。
- ③以下の通り設定し、「トピックを作成」をクリックします。
タイプ：スタンダード
名前：order-notification-topic
- ④作成したトピックを選択し、「サブスクリプション」→「作成」をクリックします。
- ⑤以下設定をします。
プロトコル：Eメール
エンドポイント：自分のメールアドレス
- ⑥⑤で設定したメールに届いた Confirm subscription をクリックして承認します。

タスク 3 : Lambda関数作成（3種）

- ①AWS Lambda コンソール を開きます。
「関数を作成」→「一から作成」を選択します。
- ②以下、設定をします。
関数名：save_order
ランタイム：Python 3.11
- ③実行ロールは「基本的な Lambda アクセス権限で新しいロールを作成」を選択し、関数を作成します。
- ④作成した関数を開き、作成されたロールのリンクをクリックします。
- ⑤IAMロールに「AmazonDynamoDBFullAccess」許可ポリシーを追加します。

⑥Lambdaの画面に戻り、下記コードに置き換えます。

```
import boto3
from datetime import datetime

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Orders")

def lambda_handler(event, context):
    order = event["detail"]
    table.put_item(
        Item={
            "orderId": order["orderId"],
            "userId": order["userId"],
            "amount": order["amount"],
            "createdAt": datetime.utcnow().isoformat()
        }
    )
    return {"status": "SUCCESS", "orderId": order["orderId"]}
```

⑦2つ目の関数を以下設定で作成します。

関数名：send_notification

ランタイム：Python 3.11

IAM ロールに AmazonSNSFullAccess を追加

※ARNは各自の設定に置き換えて下さい

```
import boto3

SNS_TOPIC_ARN = "arn:aws:sns:REGION:ACCOUNT_ID:order-
notification-topic"

sns = boto3.client("sns")

def lambda_handler(event, context):
    order = event["detail"]
    sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Subject="注文作成通知",
        Message=f"注文ID:{order['orderId']} 金額:{order['amount']}"
    )
    return {"status": "SUCCESS", "orderId": order["orderId"]}
```

⑧同様に、3つ目の関数を以下設定で作成します。

関数名：save_audit_log

ランタイム：Python 3.11

IAM ロールに AmazonDynamoDBFullAccess を追加

```
import boto3
import uuid
from datetime import datetime

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("OrderAuditLogs")

def lambda_handler(event, context):
    table.put_item(
        Item={
            "eventId": str(uuid.uuid4()),
            "createdAt": datetime.utcnow().isoformat(),
            "eventType": event.get("detail-type", "Unknown"),
            "source": event.get("source"),
            "payload": event["detail"]
        }
    )
    return {"status": "SUCCESS"}
```

タスク4：Step Functions 作成

①Step Functions コンソール を開き、「**ステートマシンを作成**」 をクリックします。

②以下設定で作成します。

作成方法：**コードで作成**

タイプ：**Standard**

IAM ロール：新規作成

③デザイン画面を「コード」に切替、以下JSONを入力します。

※3種のLambda関数のarnは各自の環境に合わせて、修正する必要があります。

```
{
  "StartAt": "ParallelTasks",
  "States": {
    "ParallelTasks": {
      "Type": "Parallel",
      "Branches": [
        {
          "StartAt": "SaveOrder",
          "States": {
            "SaveOrder": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:save_order",
              "End": true
            }
          }
        },
        {
          "StartAt": "SendNotification",
          "States": {
            "SendNotification": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:send_notification",
              "End": true
            }
          }
        },
        {
          "StartAt": "SaveAuditLog",
          "States": {
            "SaveAuditLog": {
              "Type": "Task",
              "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:save_audit_log",
              "End": true
            }
          }
        }
      ],
      "End": true
    }
  }
}
```

タスク 5 : EventBridge ルール作成

- ①EventBridge コンソール を開き、「イベントパターンを含む EventBridge ルール」 を選択し「ルール作成」をクリックします。
- ②以下設定します。
名前 : order-created-rule
イベントバス : default
- ③イベントパターンは「カスタムパターン」を選択し、以下設定をします。
名前 : order-created-rule
イベントバス : default
ルールタイプ : **イベントパターン**
- ④イベントパターン JSON を以下の通り入力します。

```
{
  "source": ["app.order"],
  "detail-type": ["OrderCreated"]
}
```
- ⑤ターゲット : **Step Functions ステートマシン** に設定
- ⑥IAM ロール : 新規作成を選択します。

タスク 6 : 動作確認

- ① EventBridge → 「イベントバス」 → default を選択します。
- ②右上の 「イベントを送信」 をクリック
- ③イベントソースと詳細タイプを入力
イベントソース : app.order
詳細タイプ : OrderCreated
- ④イベントの詳細(JSON)を以下の通り入力します。

```
{
  "orderId": "ORDER-001",
  "userId": "USER-01",
  "amount": 5000
}
```
- ⑤ 「送信」 をクリック
- ⑥Step Functions 実行履歴で ParallelTasks → 各 Lambda の成功を確認します。
DynamoDB (Orders / OrderAuditLogs) にデータが追加されていることを確認します。
SNS メール通知が届くことを確認します。

【オプション演習＜発展編＞AWS Step Functions を使用したワークフローオーケストレーション】

本演習では、サーバーレスで構築されたブックマーク共有アプリケーションを題材にします。
このアプリケーションでは、利用者が有用な Web リソース（URL）を投稿し、社内でのナレッジ共有を目的としています。

アプリケーションの利用拡大に伴い、以下のような課題が顕在化しています。

- 投稿数の増加により、手動レビューの負荷が急増
- 重複した URL や不適切な投稿が混在する可能性
- 投稿がコンテスト対象になるかどうかの判断が必要

すべてを自動化するにはリスクが高く、最終判断は人が行いたいと考えています。

マネージャーはこれらの課題を踏まえ、可能な限り 自動評価でスケールさせつつ最終的な公開判断は 人手承認として残すという、現実的で運用しやすいプロセスを求めています。

そこで、次のような投稿処理フローを段階的に構築します。

1. 利用者が Web UI からブックマークを投稿
2. 投稿内容がイベントとしてシステム内部に伝播
3. 自動チェック処理を実行
 - ・ URL の重複チェック
 - ・ 投稿内容の妥当性評価
4. 自動処理だけでは判断できない場合は**人手による承認ステップへ移行**
5. マネージャーがメールリンクから承認／却下を実施

タスク 1：DynamoDB テーブル作成

①AWS マネジメントコンソール で DynamoDB を開き、「テーブルを作成」を選択します。

②以下、設定でテーブルを作成します。

テーブル名： BookmarksTable

パーティションキー： BookmarksTable (String)

タスク 2：Lambda関数作成①

①Lambdaコンソール画面で関数名「 CreateBookmark 」、ランタイム「Python3.12」で関数を作成します。

②「 CreateBookmark 」関数の実行ロールのリンクから以下のポリシーを確認・追加します。

- ・ AWSLambdaBasicExecutionRole（CloudWatch用。存在することを確認）
- ・ インラインポリシー追加（DynamoDB操作用） アカウントID、テーブル名は置き換え。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:ap-northeast-1:<アカウントID>:table/テーブル名"
    }
  ]
}
```


③コードは以下の通りです。

```
import json
import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("BookmarksTable")
```

```
def lambda_handler(event, context):
    body = json.loads(event["body"])
```

```
    content = body["content"]
    title = body.get("title", "")
    shared = body.get("shared", False)
```

```
    try:
        table.put_item(
            Item={
                "content": content,
                "title": title,
                "shared": shared
            },
            ConditionExpression="attribute_not_exists(content)"
        )
```

```
    except ClientError as e:
        if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
            return {
                "statusCode": 409,
                "body": json.dumps({
                    "message": "Duplicate bookmark"
                })
            }
        raise
```



```
    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "OPTIONS,POST"
        },
        "body": json.dumps({
            "message": "Bookmark created successfully!"
        })
    }
```

※これにより、初回のLambdaは成功しDynamoDBへのPUTが成功、2回目以降は同じデータがPUTされることとなり409エラーとなります。

タスク3：APIGateway作成

①以下のAPIGWを作成します。

種類：HTTP API

ルート：POST /bookmarks

統合先：CreateBookmark（Lambda関数）

②以下のようにCORSの設定も加えます。

許可：Origin: *

Methods: POST,OPTIONS

Headers: content-type

③この時点での動作確認をします。①で作成したAPIGWのInvokeURLをコピーしておきます。

④PowerShellを起動し、以下コマンドを実行します。InvokeURLは置き換えること。成功メッセージが表示され、DynamoDBに項目が追加されていることを確認します。（2回目の実行は重複エラーとなります。）

```
Invoke-RestMethod `
    -Uri "<API_URL>/bookmarks" `
    -Method POST `
    -Headers @{ "content-type" = "application/json" } `
    -Body '{
        "title": "AWS EventBridge Test",
        "content": "https://example.com",
        "shared": true
    }'
```

【参考】 成功メッセージ

```
{
  "message": "Bookmark created successfully!"
}
```

タスク4：UI作成、簡易Webサーバー起動

①以下のindex.htmlを作成します。APIGWのURLはご自身のアドレスに置き換えてください。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8" />
  <title>Bookmark 投稿</title>
</head>
<body>
  <h1>Bookmark 投稿</h1>

  <label>
    URL :
    <input type="text" id="content" size="50" />
  </label>
  <br /><br />

  <label>
    タイトル :
    <input type="text" id="title" size="50" />
  </label>
  <br /><br />

  <label>
    共有する :
    <input type="checkbox" id="shared" />
  </label>
  <br /><br />

  <button onclick="submitBookmark()">投稿</button>

  <pre id="result"></pre>
```

```
<script>
  async function submitBookmark() {
    const content =
      document.getElementById("content").value;
    const title = document.getElementById("title").value;
    const shared =
      document.getElementById("shared").checked;

    const res = await fetch(
      "https://YOUR_API_ID.execute-api.ap-northeast-
      1.amazonaws.com/bookmarks",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          content,
          title,
          shared
        })
      }
    );

    const text = await res.text();
    document.getElementById("result").innerText =
      `status: ${res.status}¥n${text}`;
  }
</script>
</body>
</html>
```

②htmlファイルが保存されている場所で以下コマンドを実行し、簡易Webサーバーを起動します。

```
python -m http.server 8080
```

③ブラウザで以下アドレスにアクセスし、アプリを起動します。

<http://localhost:8080/>

④任意のデータを入力し、動作確認します。ただし重複したデータは投入できないため先ほどのテストデータとは異なるデータを入力すること。

```
status: 200
{"message": "Bookmark created successfully!"}
```

タスク5：EventBridge設定

Lambda が「Bookmark が投稿された」という事実を EventBridge に送信し、EventBridge のルールでその事実を検知したら Step Functions（ワークフロー）を起動できる状態にします。
EventBridgeでは演習専用のカスタムイベントバスを作成します。

①EventBridgeの画面を表示し、左側のメニューから「イベントバス」を選択、カスタムイベントバス欄の「イベントバスを作成」をクリックします。

②名前を「bookmarks-bus」に設定し作成します。

③Lambdaの編集をします。Lambda関数に設定されている既存ロールに以下インラインポリシーを追加し、LambdaがEventBridge に PutEvents できるよう設定します。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "events:PutEvents",
      "Resource": "arn:aws:events:ap-northeast-1:*:event-bus/bookmarks-bus"
    }
  ]
}
```

④Lambdaコードも以下を編集し、EventBridgeへ送信できるようにします。バス名は作成したEventBridgeカスタムイベントバス名に置き換えること。

```
import json
import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("BookmarksTable")

# ★追加: EventBridge クライアント
eventbridge = boto3.client("events")

def lambda_handler(event, context):
    body = json.loads(event["body"])

    content = body["content"]
    title = body.get("title", "")
    shared = body.get("shared", False)

    try:
        table.put_item(
            Item={
                "content": content,
                "title": title,
                "shared": shared
            },
            ConditionExpression="attribute_not_exists(content)"
        )

    except ClientError as e:
        if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
            return {
                "statusCode": 409,
                "body": json.dumps({
                    "message": "Duplicate bookmark"
                })
            }
        raise
```





```

# ★保存成功後のみ、shared=trueならイベント送信
if shared:
    response =eventbridge.put_events(
        Entries=[
            {
                "EventBusName": "バス名",
                "Source": "bookmark.app",
                "DetailType": "BookmarkShared",
                "Detail": json.dumps({
                    "content": content,
                    "title": title,
                    "shared": shared
                })
            }
        ]
    )
# ★追加: EventBridge送信成功ログをCloudWatch Logs出力
print("Event sent to EventBridge")
print("EventBridge response:", response)

return {
    "statusCode": 200,
    "headers": {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Headers": "Content-Type",
        "Access-Control-Allow-Methods": "OPTIONS,POST"
    },
    "body": json.dumps({
        "message": "Bookmark created successfully!"
    })
}

```

⑤EventBridgeルールを作成します。これは受けとったデータを誰に渡すのかを決めるためです。EventBridge画面の左側メニューから「ルール」を選択します。

⑥「イベントバスを選択」から作成済のバスに切り替え、「ルールを作成」ボタンをクリックします。
 ※ここで選択するバスは、Lambda関数コード内の「"EventBusName": "バス名"」と一致している必要があります。
 ルール名に「BookmarkCreatedRule」、カスタムバスを確認し、「次へ」をクリックします。

⑦イベントは「その他」を選択、イベントパターンは「カスタムパターン（JSONエディタ）」に変更し、以下のJSONを入力し、「次へ」をクリックします。

```

{
    "source": ["bookmark.app"],
    "detail-type": ["BookmarkShared"]
}

```

⑧ターゲットは未作成のため、ダミーでLambda関数を選択し、任意の関数に設定しルール作成を完了します。（ターゲットは後で修正）

⑨ここまでの動作確認をします。アプリを起動し、データを投入します。
 LambdaのCloudWatchLogsを確認し、「Event sent to EventBridge」「EventBridge response」とログが生成されていることを確認します。

タスク6：Step Functions（承認ワークフロー）作成

EventBridge に「BookmarkCreated」という事実が伝わり、次に業務フローをStep Functionsによって制御します。条件によって処理を分岐し、人の手による承認作業および結果確定を実現し、流れを可視化します。

①Step Functionsの画面を表示し、これまで利用していたリージョンになっていることを確認します。

②左側のメニューから「ステートマシン」を選択、「ステートマシンの作成」ボタンをクリックします。

③「空白から作成」を選び、任意の名前を入力、タイプは「標準」を選択し続行します。

④「デザインモード」になっている画面を「コード」に切替、既存コードを削除し、以下JSONデータを入力し、作成します。

※コード内で指定しているLambda関数「 CheckDuplicateBookmark 」は次の手順で作成します。

※作成画面で表示されるアクセス許可は変更せず、作成します。

```
{
  "StartAt": "CheckDuplicate",
  "States": {
    "CheckDuplicate": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName": "CheckDuplicateBookmark",
        "Payload.$": "$"
      },
      "ResultPath": "$.duplicateResult",
      "Next": "IsDuplicate?"
    },
    "IsDuplicate?": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.duplicateResult.Payload.isDuplicate",
          "BooleanEquals": true,
          "Next": "EndDuplicate"
        }
      ],
      "Default": "RequestApproval"
    },
    "EndDuplicate": {
      "Type": "Fail",
      "Error": "DuplicateBookmark",
      "Cause": "Bookmark already exists"
    },
    "RequestApproval": {
      "Type": "Pass",
      "End": true
    }
  }
}
```

⑤先ほどダミー設定をしていたEventBridgeのルールターゲットを修正します。EventBridgeのルール画面を表示し、作成済のルールを編集します。

⑥ターゲットタイプ：Step Functionsのステートマシンを設定し、作成済のステートマシンを設定、ルールを更新します。

⑦ CheckDuplicateBookmark Lambda関数を作成します。これは登録するブックマークがすでに登録されているかを判定する関数で、判定結果の真偽値のみ返します。

関数名： CheckDuplicateBookmark（Step Functionsで設定した名前を一致していること）

ランタイム：Python3.12

アクセス許可：インラインポリシー作成

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:Query",
        "dynamodb:Scan"
      ],
      "Resource": "arn:aws:dynamodb:ap-northeast-1:*:table/テーブル名"
    }
  ]
}
```

コード：

```
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("テーブル名")

def lambda_handler(event, context):
    """
    event は EventBridge 由来の入力をそのまま受け取る
    Step Functions からは Payload.$ = "$" で渡される前提
    """

    # EventBridge → detail → content
    content = event.get("detail", {}).get("content")

    if not content:
        # 判定不能だが「重複ではない」として先に進める
        return {
            "isDuplicate": False,
            "reason": "content missing"
        }

    response = table.scan(
        FilterExpression="content = :content",
        ExpressionAttributeValues={
            ":content": content
        }
    )

    return {
        "isDuplicate": response["Count"] > 0
    }
```

⑧メール送信のためSNSを設定します。Amazon SNSの画面から「トピックの作成」へ進みます。

⑨タイプ：スタンダード、名前：BookmarkNotificationとしトピック作成します。

⑩自身のメールアドレスでサブスクリプションを作成し、受信したメールから承認を行います。
TopicのARNを控えておいてください。

⑪承認メール送信 Lambda関数を作成します。

関数名： SendApprovalMail

ポリシー： インラインポリシー追加（TopicのARNは置き換えること）

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:ap-northeast-1:xxxxxxxxxxx:BookmarkNotification"
    }
  ]
}
```

コード： ※TopicのARNは置き換えること

※「 approve_url 」はダミーです。後ほど修正します。

```
import json
import boto3
import urllib.parse

sns = boto3.client("sns")

TOPIC_ARN = "arn:aws:sns:ap-northeast-1:xxxxxxxxxxx:BookmarkNotification"

def lambda_handler(event, context):
    task_token = event["TaskToken"]
    detail = event["detail"]

    approve_url = (
        "https://example.com/approve?"
        + urllib.parse.urlencode({"token": task_token})
    )

    message = f"""
Bookmark approval required.

Title: {detail.get('title')}
URL: {detail.get('content')}

Approve:
{approve_url}
"""

    sns.publish(
        TopicArn=TOPIC_ARN,
        Subject="Bookmark Approval Required",
        Message=message
    )

    return {}
```

⑫ステートマシンに承認ステートを追加します。JSONのコードを以下に修正し、デザイン画面が変更されたことを確認します。

```
{
  "StartAt": "CheckDuplicate",
  "States": {
    "CheckDuplicate": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName": "CheckDuplicateBookmark",
        "Payload.$": "$"
      },
      "ResultPath": "$.duplicateResult",
      "Next": "IsDuplicate?"
    },
    "IsDuplicate?": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.duplicateResult.Payload.isDuplicate",
          "BooleanEquals": true,
          "Next": "EndDuplicate"
        }
      ],
      "Default": "SendApprovalMail"
    },
    "SendApprovalMail": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke.waitForTaskToken",
      "Parameters": {
        "FunctionName": "SendApprovalMail",
        "Payload": {
          "TaskToken.$": "$$.Task.Token",
          "detail.$": "$.detail"
        }
      },
      "Next": "Approved"
    },
    "Approved": {
      "Type": "Pass",
      "End": true
    },
    "EndDuplicate": {
      "Type": "Fail",
      "Error": "DuplicateBookmark",
      "Cause": "Bookmark already exists"
    }
  }
}
```

⑬Callback受信用のLambda関数を作成します。
これはメールリンクから人間による承認操作を
StepFunctionsに伝えるための関数です。

関数名: ApprovalCallback

インラインポリシー追加:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "states:SendTaskSuccess",
        "states:SendTaskFailure"
      ],
      "Resource": "*"
    }
  ]
}
```


コード:

```
import json
import boto3
import urllib.parse

sf = boto3.client("stepfunctions")

def lambda_handler(event, context):
    try:
        # ===== クエリパラメータ取得 (HTTP API v2
        # 対応) =====
        params = event.get("queryStringParameters") or {}

        task_token = params.get("token")
        action = params.get("result", "approve")

        if not task_token:
            raise Exception("taskToken is missing")

        # URLデコード (必須)
        task_token = urllib.parse.unquote(task_token)

        if action == "approve":
            sf.send_task_success(
                taskToken=task_token,
                output=json.dumps({"approved": True})
            )
        else:
            sf.send_task_failure(
                taskToken=task_token,
                error="Rejected",
                cause="Rejected by reviewer"
            )
```

```
        return {
            "statusCode": 200,
            "body": "Approval processed successfully"
        }

    except Exception as e:
        print("ERROR:", str(e))
        return {
            "statusCode": 500,
            "body": json.dumps({"message": "Internal
Server Error"})
        }
```

⑭ApprovalCallback 用の API Gateway を作成します。
Amazon API Gatewayの画面でHTTP API を構築します。

API名: 任意
統合タイプ: Lambda
Lambda関数: ApprovalCallback
ルート設定: GETメソッド
パス: /approve

⑮⑭で作成したLambda関数の情報で「SendApprovalMail」関数の中身を修正し、再デプロイします。
修正前: "https://example.com/approve?"

修正後:

```
APPROVAL_API_URL = "https://xxx.execute-api.ap-northeast-1.amazonaws.com/approve"

approve_url = (
    APPROVAL_API_URL + "?"
    + urllib.parse.urlencode({
        "token": task_token,
        "result": "approve"
    })
)
```

⑩最終の動作確認をします。
ブラウザでアプリを起動し、任意のデータを入力します。
(共有する；オン)

Bookmark 投稿

URL：

タイトル：

共有する：☐

投稿

⑪ブラウザ上に成功メッセージが表示されることを確認します。

Bookmark 投稿

URL：

タイトル：

共有する：☒

投稿

```
status: 200
{
  "message": "Bookmark created successfully!"
}
```

⑫設定した自身のメールアドレスにメールが届いています。
現在、承認待ちの状態です。

Bookmark Approval Required

Bookmark approval required.

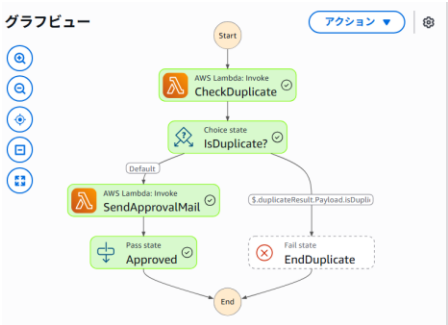
Title: test.test
URL: test.test

Approve:
[https://rb1xhnpn4.execute-api.ap-northeast-1.amazonaws.com/approve?](https://rb1xhnpn4.execute-api.ap-northeast-1.amazonaws.com/approve?token=AQB4AAAKgAAAAAAAAAAAAAAAAAajMmE33ldvyQABGICgntPpYXWWOJ4zmpZIEK%2BrU)
[token=AQB4AAAKgAAAAAAAAAAAAAAAAAajMmE33ldvyQABGICgntPpYXWWOJ4zmpZIEK%2BrU](https://rb1xhnpn4.execute-api.ap-northeast-1.amazonaws.com/approve?token=AQB4AAAKgAAAAAAAAAAAAAAAAAajMmE33ldvyQABGICgntPpYXWWOJ4zmpZIEK%2BrU)

⑬受信メールのリンクをクリックします。リンクをクリックしたことで承認されたとなり、メール閲覧画面（ブラウザ）に以下メッセージが表示されれば成功です。

Approval processed successfully

EventBridgeのルール画面のモニタリングでルールのマッチ数の確認やStepFunctions画面のステートマシンで「実行」タブでフローの状況が確認できます。



09 : オブザーバビリティと モニタリング

Chapter 9

■ モダンアプリケーションで、従来のモニタリングアプローチに挑む



存続期間の
短いリソース



その他のデバイス、
サービス、
データ



リリースの
高速化



ユーザー
エクスペリエンス

モダンアプリケーション、特にサーバーレスのアプリケーションでは、存続期間が短いリソースや、分散した方法で接続されたさまざまなサービスについて考慮する必要があります。また、独立したコンポーネントのリリース速度の高速化についても検討する必要があります。ユーザーエクスペリエンスは、アプリケーションのパフォーマンスにおける重要な考慮事項にもなっています。例えば、ユーザーは低速な接続を許容しません。このタイプのモニタリングでは、スタック内のレイヤーの1つのエラーに関するアラートとは異なるアプローチが必要です。

■ モニタリングでしていたこと:

- スタックのレイヤーを見る
- 調査、クエリ、レポートにより障害を特定する
- メトリクスにフォーカス

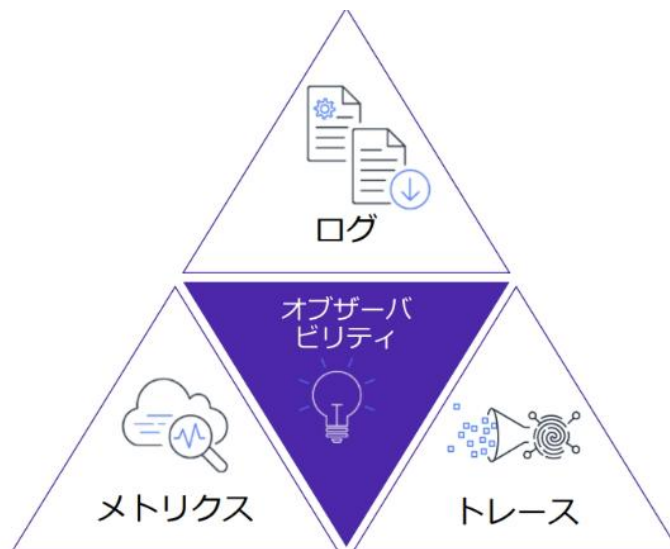
■ オブザーバビリティ:

- システム全体を可視化し、期待どおりに動作しているかどうかを把握する
- インサイトを得る
 - 利用額
 - カスタマーエクスペリエンス
 - 傾向
 - 根本原因

これまでのモニタリングでは、スタックのレイヤー全体で障害を特定することに焦点を当てていましたが、既知の障害がないことのみを示すダッシュボードでは、すべてを把握することはできませんでした。アプリケーションは期待通りに動作していますか？使用方法とアクセスパターンはどのようなものですか？アプリケーションのどの部分が、設計で意図したよりも高い頻度または低い頻度で使用されていますか？システムに関するビジネス情報も必要です。その情報は収益を生み出していますか？成長が見られるのはどこですか？どのような傾向が見られ、それに対して何を計画できますか？

オブザーバビリティは、障害を監視する以上のことを行うために、システムに関するデータの収集と使用についての考え方を推進する組織文化と開発手法へのアプローチを反映しています。オブザーバビリティは、従来のモニタリングを拡張し、アプリケーションに関する疑問に対処するアプローチを提供します。ビジネスのメトリクスは後から付け足されることがあり、ビジネスの誰かが質問をしたときにのみ機能します。自分が持っているデータから答えを得る方法を理解する必要があります。しかし、アプリケーションを構築するときにこれらのニーズを組み込むめば、アプリケーション内で起きていることを可視化できます。

■ オブザーバビリティの 3 つの柱はログ、トレース、メトリクス



ログ、トレース、メトリクスは、オブザーバビリティの 3 つの柱です。

- ログは、障害、エラー、状態変化などの個別イベントのタイムスタンプ付きレコードです。ログには、メトリクスが含まれる場合があります。
- トレースでは、個別のアプリケーションまたはアプリケーションの各部分を通じて、ユーザーのジャーニーの 1 つのトランザクションがわかります。
- メトリクスは、間隔ごとに測定された数値データを提供します。多くの場合、リクエスト率やエラー率などのサービスレベルのメトリクスですが、受注数や支払い数などのビジネスのメトリクスになることもあります。

これらの 3 つの柱は、システムのオブザーバビリティを達成するために、文脈に応じてリンクする必要があります。例えば、メトリクスに関するアラートを受け取った場合、メトリクスに関連するログをすばやく見つけて、イベントに関連付けられたトレースを詳細に調べることができます。

オブザーバビリティアプローチでは、障害はアプリケーションの構築後にモニタリングされるだけでなく、活用されます。障害モードは、ソフトウェア開発のライフサイクルの中で、アプリケーションの一部として設計されています。

システムを実際に理解するのに十分なデータが収集できていることを確認してください。データが足りない場合は、可視性にギャップが生じる可能性があります。

重要なのは、アプリケーションの構築中にデバッグに必要なデータを組み込むことです。また、カナリアなどの本番テストメカニズムを導入プロセスに組み込むようにします。

■ 開発者向け機能

- ログを時間順に並べたイベントのフローとして確認する
- CloudWatch Logs Insights を使用して、ログデータを対話的にクエリできる
- メトリクスフィルターでログデータを使用し、アラームを設定する
- 追加の分析または処理のためにログデータをエクスポートする

Amazon CloudWatch Logs を使用すると、ログを確認およびクエリすることができます。メトリクスアラームを使用してログデータを処理したり、追加のユースケース用にデータをエクスポートすることもできます。

Amazon Simple Storage Service (Amazon S3) にエクスポートし、Amazon Data Firehose 経由で Amazon OpenSearch Service (OpenSearch Service) にストリーミングし、OpenSearch Dashboards などのツールを使用してログを分析したり、サードパーティー製のツールにエクスポートして分析や処理を行ったりすることができます。

- デバッグ用の CloudWatch Logs (REST のみ)
 - エラーレベルまたは情報レベルを選択する
 - リクエスト/レスポンスデータ全体のログ記録を有効化するオプション
 - メソッドごとのステージレベル設定を上書きするオプションで、API ステージごとに有効化する
 - ベストプラクティス: 本番環境における冗長性の低いオプションでのトラブルシューティングとデバッグに使用する
- アクセスログ (REST および HTTP)
 - 有効にすると、API リクエストごとに書き込まれる
 - カスタマイズ可能なフォーマット
 - ベストプラクティス: 使用可能なテンプレートを開始点として使用して、アクセスのログ記録を有効にするアクセスログを使用して、オーソライザーのレスポンスを確認する

デフォルトでは、API Gateway で API を作成する場合、ログ記録は設定されません。

ログ記録の動作は、REST と HTTP の API で少し異なります。

一般的なベストプラクティスとして、アクセスログを有効にし、提供されているテンプレートの 1 つから開始します。少なくとも、ログ記録を使用して認証者の応答を確認する必要があります。

Lambda のログ記録

ロググループ ログストリーム

Cloudwatch > Log Groups > /aws/lambda/FAQ > 2020/08/27/[\$LATEST]05419d29f9014fc3b1634b1e9968ef28

時間	メッセージ
19:35:32	START RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Version: \$LATEST
19:35:32	2020-08-27T19:35:32.553Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO ...
19:35:32	2020-08-27T19:35:32.575Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO ...
19:35:32	END RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937
19:35:32	REPORT RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Duration: 64.40 ms Billed Duration: 100 ms Memory Size: 128 MB Max Memory Used: 68 MB Init Duration: 162.85 ms
19:35:36	START RequestId: a23cbea6-3053-4b51-8721-ff11aba24e33 Version: \$LATEST
...	

ログイベント

Lambda ログは、すべてのリクエストと結果を /aws/lambda/<function name> という CloudWatch ロググループに記録します。関数とロググループの間には 1 対 1 の関係が存在します。関数を初めて実行すると、グループが作成されます。プロビジョニング済みの同時実行を設定した場合、プロビジョニング済みの同時実行の作成時にロググループが作成されます。ロググループは、Lambda ログ内のイベントを検索するために通常使用するエンティティです。

ロググループは、日付、関数バージョン、および一意の識別子によって識別されるログストリームで構成されます。Lambda は、環境への呼び出しの割り当て方法および呼び出しのタイミングに関連する内部要因に基づいて、グループ内に新しいログストリームを自動的に作成します。

各ログストリームには、Lambda リクエストの詳細を記述するイベントのシーケンスが含まれます。各リクエストには一意の RequestID が含まれ、START イベント、END イベント、および REPORT イベントが含まれます。これらのイベントは、Lambda リクエストに関する概要情報をすばやく通知します。

RequestId は、特定の呼び出しに関連するイベントを検索するために使用するプライマリ識別子です。REPORT イベントでは、関数の実行時間、課金期間、関数に設定されたメモリ、および関数が使用したメモリをすばやく確認することができます。この例の「Init」期間は、この呼び出しがコールドスタートであったことを示しています。

`stdout` または `stderr` に書き込んで追加の詳細を記録する

例:

- Node.js: `console.log('write this to the log')`
- Python: `print("write this to the log")`

ランタイムコマンドまたは `stdout` または `stderr` に書き込むログ記録ライブラリを使用します。

■ Lambda ログを構造化して、読みやすさとレポートを改善する

```
message = {  
    "PriceInCart": 100,  
    "QuantityInCart": 2,  
    "ProductId": "a23390f3",  
    "CategoryId": "bca4cec1",  
    "UserId": "31ba3930",  
    "CartId": "58dd189f",  
    "Environment": "prod",  
    "LogLevel": "INFO",  
    "Timestamp": "2019-12-11 12:44:40.300473",  
    "Message": "Added 2 items 'a23390f3' to  
    cart '58dd189f'"  
}
```

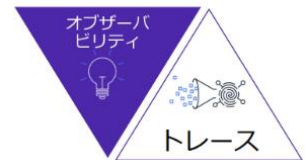
Python:
`print(json.dumps(message))`

Node.js:
`console.log(JSON.stringify(message));`

- 共通のログデータのフィールドを自動的に生成する
(例えば @message、@timestamp)
- ログ内の構造化されたフィールドを検出する
- データを解析して、クエリのエフェメラルフィールドを作成できる
- 多数のサービスに対して事前作成されたクエリを提供する

■ 開発者向け機能

- ワンクリックでの Lambda 関数 の有効化と、他の AWS のサービスとの統合
- X-Ray SDK では、AWS SDK を使用してAWS のサービスに対して実行された API コールのメタデータをキャプチャする
- レイテンシー分布の視覚的な検出と外れ値および傾向の迅速な分離
- アプリケーションコードのデバッグが容易
- エラータイプによるリクエストのフィルタリングとグループ化



他の AWS のサービスは、次のいずれかの統合方法を使用して X-Ray と統合します。

- アクティブ計測は、受信リクエストをサンプリングして計測します。
- パッシブ計測は、別のサービスですでにサンプリングされているリクエストを計測します。
- リクエストのトレースは、すべての受信リクエストにトレースヘッダーを追加してダウンストリームに伝達します。
- ツールは、AWS X-Ray デーモンを実行して X-Ray SDK からセグメントを受信します。

Lambda などの一部のサービスでは、サービスで X-Ray を有効にするだけで、X-Ray は新しいリクエストのサンプリングとトレース情報の追加を開始します。これはアクティブ計測です。

その他のサービスでは、サービスに渡されたトレースヘッダーを取得し、それを生成したサービスのトレース ID を使用してトレースデータを伝播できます。これにより、リクエスト全体で継続性が得られるため、下流サービスを通じてトレースを追跡することができます。これはパッシブ計測です。

アクティブトレースとパッシブトレースの両方で、新しいセグメント ID が生成されますが、パッシブトレースでは、トレース ID は同じままです。トレース ID がまだない場合、新しいトレース ID は生成されません。

X-Ray SDK を使用して AWS SDK 経由で AWS のサービスに対して実行された API コールのメタデータをキャプチャすることができます

■ X-Ray によって詳細が要約され、トラブルスポットが示される



トレース リクエスト

X-Ray は、通過する基盤となるサービスからリクエストに関するデータを収集し、データに ID のタグを付ける



レコード トレース

X-Ray がセグメントとしてサービスからデータを取得し、リクエストごとにデータをグループ化およびフィルタリングできるトレースへとグループ化する



サービスマップの 確認

X-Ray は、クリック可能なマップを介してトレースを要約し、レイテンシー、HTTP ステータス、メタデータなどを強調表示する



トレース詳細への ドリルダウン

問題を特定できるように、X-Ray が各リクエストについてセグメント化された内訳を示す

X-Ray を使用すると、リクエストに関するトレースデータをキャプチャするようにサービスを設定することができます。一部のサービスでは、X-Ray を直接有効化できます。

X-Ray は、リクエストを処理する各サービスからのトレースデータを、トレースと呼ばれる単位に結合します。

X-RAY サービスマップには、システム内を移動するリクエストの概要が表示され、トラブルスポットが示されます。サービスマップから、指摘されたトラブルスポットをドリルダウンして、X-Ray がキャプチャした各トレースセグメントに関するデータを確認できます。各アクティビティにかかった時間を確認することができ、エラーのあるセグメントが X-Ray により表示されます。

X-Ray は、コード内およびダウンストリームサービスの 3 種類のエラーを追跡します。

- エラー: クライアントエラー (400 シリーズ)
- 障害: サーバー障害 (500 シリーズ)
- スロットル: スロットリングエラー (429 Too Many Requests)

■ トレースの各部分

要素	説明
セグメント	アプリケーションを実行するコンピューティングリソースによって行われた処理に関するデータ: ・ リソース名 ・ リクエストと完了した処理の詳細 ・ サブセグメント
サブセグメント	セグメントデータのより詳細な内訳: ・ より詳細なタイミング ・ ダウンストリームコールの詳細 ・ ダウンストリームサービスの推定セグメント ・ 注釈とメタデータ
注釈	インデックスを付け、フィルター式で使用するコンソールでトレースをグループ化し、分析を容易にすることができるキーと値のペア
メタデータ	インデックス付けされないが、トレースの検索に使用しないデータをトレースに格納するために使用できる任意のタイプのキーと値のペア

セグメントには、リソース名、リクエストの詳細、行った作業の詳細が含まれています。セグメントでは、完了した作業に関するデータをサブセグメントに分けることができます。サブセグメントには、詳細なタイミング情報や、元のリクエストを満たすためにアプリケーションが実行したダウンストリーム呼び出しに関する詳細が含まれます。サブセグメントには、AWS のサービス、外部 HTTP API、SQL データベースに対する呼び出しに関する追加の詳細を含めることができます。アプリケーションのコードの特定の関数または行を計測する任意のサブセグメントを定義することもできます。

注釈とメタデータの追加データにより、フィルター式の使用や論理グループ内のトレースの確認が容易になります。

■ 開発者向け機能

- AWS のサービスのデフォルトのメトリクスを使用する
- メトリクスのしきい値に基づいてアラームを設定し、自動アクションを開始する
- ログとメトリクスを簡単に関連付けることができる
- カスタムメトリクスを作成する
- 埋め込みメトリクス形式を使用してログファイルからメトリクスを作成する



[CloudWatch Logs] セクションを強調表示すると、ログデータに基づいてメトリクスを作成することができます。また、CloudWatch コンソールと CloudWatch Logs Insights を使用して、サービスに関連するログとメトリクスの間を簡単に移動できます。

ビジネスニーズ（カスタマーアクティビティなど）に対応するカスタムメトリクスを作成できます。また、しきい値を超えたときに通知したり、その他のアクションを実行したりするアラームを設定することができます。

■ デフォルトの運用メトリクスの使用によるパフォーマンスの監視

ツール	説明
サービスコンソール	一部のサービスコンソールからメトリクスを直接表示する • Lambda: 関数の [モニタリング] タブ • API Gateway: REST API ダッシュボード • Amazon SQS: キューの [モニタリング] タブ
CloudWatch メトリクスコンソール	サービス、メトリクス、期間を選択し、グラフをカスタマイズする
カスタムダッシュボード	グラフ化されたメトリクスをダッシュボードに追加して、リソースの単一のビューをカスタマイズする

AWS のサービスではさまざまな組み込みメトリクスを使用することができます。一部のサービスでは、サービスコンソールに直接メトリクスダッシュボードが表示されます。

サービスコンソールからメトリクスを確認するときは、メトリクスグラフタイトルの横にある [...] アイコンを使用して、同じメトリクスの CloudWatch メトリクスコンソールビューにジャンプします。

CloudWatch メトリクスコンソールから、サービス、グラフ化する特定のメトリクス、および期間を選択し、メトリクスに合うようにグラフスタイルをカスタマイズできます (例えば、集計値と傾向を示す折れ線グラフなど)。

再利用するビューを作成する場合は、そのビューをダッシュボードに追加できます。カスタムダッシュボードに複数のビューを追加したり、異なるサービスのビューを追加したりすることができます。また、ダッシュボードを他のユーザーと共有することもできます。

CloudWatch Lambda Insights

- Lambda 関数の高度なログ記録オプションを含む
- パフォーマンスメトリクス、エラー、ログの収集と操作を簡素化し、パフォーマンスの問題の切り分けを行う
- 関数ごとに有効化する Lambda レイヤーを使用する

■ アラームを使用してメトリクスにアクションを実行する

- CloudWatch アラームを使用して、CloudWatch メトリクスを追跡し、しきい値に対する相対値に基づいてアクションを実行する

- 例:

- API Gateway 5XXError: 例外ダウンストリームまたはゲートウェイサービスの問題によりサーバー側でエラーが発生したリクエストの数。これは、迅速に対応する必要があるメトリクスである
- Lambda Errors: タイムアウト、例外、またはサービスの問題によって失敗した関数の呼び出しの数。アラームにはコストがかかるため、すべての機能でアラームを設定する必要はないが、重要な機能では、このアラームが重要になる場合がある

アラームを使用すると、CloudWatch メトリクスを追跡し、しきい値を基準とした値に基づいてアクションを実行できます。例えば、Amazon Simple Notification Service (Amazon SNS) のトピックを使用して、アラームの状態が変わったときにメールを送信したり、Amazon EventBridge のルール (以前は Amazon CloudWatch Events) を使用してアラームに基づいてアクションを実行したりすることができます。しきい値として値を使用する静的アラームを設定することも、異常検出を使用してモデル化されたデータを使用してしきい値を決定することもできます。

メトリクスのアラーム状態は、アラーム条件の有無、条件を評価するのにデータが不十分でないか、すべてが正常であるかを示します。

多くの潜在的なメトリクスがアラームに対して意味を成します。いくつかの例を次に示します。

API Gateway

5xxError: 例外ダウンストリームまたはゲートウェイサービスの問題によるサーバー側のエラーがあるリクエストの数です。これは、迅速に対応する必要があるメトリクスです。

Lambda

Errors: タイムアウト、例外、またはサービスの問題によって失敗した関数の呼び出しの数です。アラームにはコストがかかるため、すべての機能でアラームを設定する必要はありませんが、重要な機能では、このアラームが重要になる場合があります。

IteratorAge: ストリームベースの呼び出し専用のメトリクスです。処理されたレコードのバッチごとに最後のレコードの経過日数を測定します。値が増加している場合は、ストリームでシャードが停止したことを意味します。

監視したいが、アラームをオンにする必要がないメトリクスについては、ダッシュボードを作成することができます。例えば、同時実行呼び出しと予約されていない同時呼び出しは、追跡または確認に適したメトリクスですが、アラームは必要ありません。

■ ビジネスメトリクス

- お客様の使用状況
- 注文数
- カート内のものの
コンバージョン率
- ログインしている人

JavaScript のカスタムメトリクスコードの例

```
var params = {
  MetricData: [
    {
      MetricName: 'visits',
      Timestamp: new Date || 'Wed Dec 18 2019 16:00:00 GMT-0800 (PST)',
      Unit: 'Count',
      Value: '2.00',
    },
  ],
  Namespace: 'myDiner-app'
};
cloudwatch.putMetricData(params, function(err, data) {
  if (err) console.log(err, err.stack); // an error occurred
  else console.log(data); // successful response
});
```

アプリケーションコードを使用してカスタムメトリクスを追加すると、以前に学習したビジネスメトリクスのタイプを取得できます。例えば、アプリケーションを使用している人数、アクセスパターン、受け取った注文数、カートに商品を入れて実際にトランザクションを完了した人のコンバージョン率などのメトリクスを追加することができます。

ランタイムに SDK を使用してコードを記述します。

API コールごとに料金が発生するため、これを大規模に実行すると、コストが大幅に増加します。API へのコールは同期呼び出しであるため、関数はその完了を待機します。これは瞬時に行われますが、関数を実行するのにかかる時間が長くなります。putMetricData API の呼び出しにかかる時間は、関数全体の実行にかかる時間よりも長くなる可能性があるため、メトリクスの書き込みには関数の実行よりも多くのコストがかかります。代わりに、埋め込みメトリクス形式を使用してメトリクス情報をログに追加し、CloudWatch に利用可能なメトリクスとして使用させることができます。

■ 埋め込みメトリクス形式を使用したログからのメトリクスの自動生成

- Lambda 関数などの一時的なリソースから、実用的なカスタムメトリクスを生成する
- CloudWatch を使用してカスタムメトリクスを自動的に抽出する
- CloudWatch Logs Insights をクエリに使用する

```
{
  "_aws": {
    "Timestamp": 1574109732004,
    "CloudWatchMetrics": [
      {
        "Namespace": "lambda-function-metrics",
        "Dimensions": [ ["functionVersion"] ],
        "Metrics": [
          {
            "Name": "time",
            "Unit": "Milliseconds"
          }
        ]
      }
    ]
  },
  "functionVersion": "$LATEST",
  "time": 100,
  "requestId": "989ffbf8-9ace-4817-a57c-e4dd734019ee"
}
```

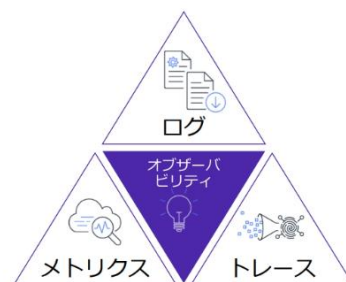
埋め込みメトリクス形式を使用すると、CloudWatch はログストリームを参照し、メトリクスを取得します。その後、アラームやダッシュボードで取得したメトリクスを使用することができます。

また、CloudWatch Logs Insights からこれらのメトリクスフィールドに関するクエリを実行するために、ログ内のデータを利用することもできます。

■ X-Ray トレースマップで3つの柱をまとめる

■ 開発者向け機能

- X-Ray と CloudWatch を統合
- ログ、メトリクス、アラームへの統合アクセスが可能なトレースマップを含む
- トラブルシューティングのためのワンストップショップを提供



【演習4 オブザーバビリティとモニタリング】

タスク1： Postman設定

- ①**Postman ダウンロード** をダウンロードします。（ <https://www.postman.com/downloads/> を開きます）
- ②**Windows (64-bit)** を選択し、インストーラ (.exe) をダウンロードします
- ③ダウンロードした .exe を実行し、画面の指示に従ってインストールします。
- ④一部機能を使用するためにアカウント登録が必要なため自身のメールアドレスを使用し登録します。（無料プランでOK）
- ⑤Postman起動時にWindowsファイアウォールの許可を求められた場合は、許可を選択してください。

タスク 2：Lambda 関数作成

- ①AWS マネジメントコンソール からLambdaを開きます。
- ②「関数の作成」をクリックします。
- ③「一から作成」を選択し、以下を入力します：
関数名：orders-api-handler
ランタイム：Python 3.11
実行ロール：「基本的な Lambda アクセス権限で新しいロールを作成」
- ④「関数を作成」をクリックします。
- ⑤作成した関数のコードタブに、以下をコピーして置き換え、デプロイします。

```
import json
import time
import random
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    request_id = context.aws_request_id
    logger.info(f"request_id={request_id} start")

    rand = random.random()

    # 5% の確率でエラーを発生
    if rand < 0.05:
        logger.error(f"request_id={request_id} simulated error")
        raise Exception("Simulated application error")
```

```
# 一部リクエストのみ遅延
if rand < 0.20:
    delay = 0.8
else:
    delay = 0.1

time.sleep(delay)
logger.info(f"request_id={request_id} delay={delay}")

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "Order information was successfully
retrieved",
        "delay": delay,
        "requestId": request_id
    })
}
```

タスク 3：HTTP API 作成

- ①AWS マネジメントコンソール からAPI Gatewayを開きます。
- ②「HTTP API を作成」をクリックします。
- ③「構築方法」は「一から」を選択し、「次へ」をクリックします。
- ④名前：「orders-api」と入力、APIを作成します。
- ⑤統合画面で「Lambda 関数」を選択し、先ほど作成したorders-api-handlerを選択します。
- ⑥「統合の追加」をクリックします。（Lambda プロキシ統合はオンのままで OK）
- ⑦左ペインで「ルート」を選択します。
- ⑧「ルートの作成」をクリックします。
- ⑨メソッド「/orders/{orderId}」を作成します。
※HTTPAPIは自動デプロイのためデプロイ操作不要です。

タスク 4：ログとトレース有効化

- ①CloudWatch コンソールを開き、左ペインで「Log管理」を選択し、「ロググループの作成」をクリックします。
- ②ロググループ名に「orders-api-prod」と名前を入力し、「作成」をクリックします。
- ③API Gateway コンソールで 作成済のAPIを選択し、左ペインの「Metrics」を選択します。
- ④ステージの選択で「\$default」を選び、「デフォルトのルートメトリクス」の「編集」をクリックし、「詳細ルートメトリクス」をオンにします。
- ⑤同じく左ペインの「Logging」を選択し、「編集」をクリックします。
- ⑥「アクセスのログ」をオンにし、「ログの送信先」に作成済のロググループのARNを指定します。
- ⑦「ログの形式」はJSONを選択し、以下を入力、「保存」します。

```
{ "requestId": "$context.requestId", "status": "$context.status", "integrationLatency": "$context.integrationLatency" }
```

- ⑧Lambdaコンソールの作成済Lambdaの設定メニューを選択します。
- ⑨「設定タブ」、「モニタリングおよび運用ツール」から「その他の監視ツール」の編集を選択、「Lambda サービストレース」を有効化にし、保存します。

タスク 5：負荷テスト

- ①Postman を開き、「新規リクエスト」をクリックします。
 - ②メソッドで「GET」を選択し、URL にAPI Gatewayステージの Invoke URL + /orders/ORDER-001
を入力します。
(例：https://XXXXXXX.execute-api.ap-northeast-1.amazonaws.com/orders/ORDER-001)
 - ③「送信」をクリックし、レスポンス 200（成功）が返り、messageやrequestIdが表示されることを確認します。
 - ④複数リクエストを短時間に送信し、負荷をかけます。
Postmanの左ペインの Collections（コレクション）を選択します
 - ⑤使用したいコレクションの右側にある「…（縦の3点リーダー）」をクリックします
 - ⑥表示されるメニューから「実行」をクリックすると右側にパネルが表示されます。
 - ⑦設定画面で「Iterations」に「50」、「Delay」に「100（ms）」と設定します。
※これにより合計で 50 リクエストを短時間で送信可能となり、ランダム遅延と 5% エラーが起こり、ピーク時の SLA 違反兆候を再現します。
 - ⑧CloudWatch画面の左ペインから「すべてのメトリクス」を選択し、「APIGateway」をクリックします。
 - ⑨本演習のAPIを選択し、以下メトリクスが存在することを確認します。
 - ・ Latency（レイテンシー）
 - ・ 4XX/5XX エラー率
 - ⑩X-Ray画面を開き、対象期間を選択します。
 - ⑪API Gateway + Lambda のトレースマップが表示されます。これにより以下を確認できます。
 - ・ API Gateway 側の遅延
 - ・ Lambda 実行時間
 - ・ エラー発生箇所
- ※今回はLambda関数のコードで作為的に負荷をかけましたが、本番環境では以下のようなポイントが分析できます。
- レスポンス遅延がランダム → Lambda 内の遅延が原因
 - 5% のエラー発生 → Lambda 内での擬似例外が原因
 - ピーク時のレイテンシー上昇 → Lambda 同時実行制限や API Gateway スロットリングが影響
- X-Ray トレースでボトルネック箇所を特定し、改善策を検討

タスク 6：通知メール設定

しきい値を超えた場合、リアルタイムでメールによる通知が届くようにCloudWatch AlarmとSNSを組み合わせます。

①Postman画面の左側からコレクション名を選択し、右側の「スクリプトタブ」の「pre-request」欄に以下のコードを入力します。

※orderIdをランダムに生成し、それをコンソールに出力するため。

```
pm.variables.set("orderId", Math.floor(Math.random() * 1000) + 1);  
console.log("orderId:", pm.variables.get("orderId"));
```

②SNSトピックを作成します。ご自身のアドレスを設定し、必ずconfirmをしてください。

③CloudWatch Alarm 作成を作成し、SNSトピックと連携します。

<監視①：Lambda エラー数>

メトリクス

Namespace：AWS/Lambda

Metric：Errors

統計：Sum（合計）

期間：1 分

しきい値：>= 1

<監視②：Lambda実行時間（遅延検知）>

メトリクス

Metric：Duration

統計：Average

期間：1 分

しきい値：>700（ms）

④Postmanで負荷をかけて実行します。コレクション名の3点リーダーから実行し、以下負荷設定をします。

Iterations：300 （300 リクエスト）

Delay：0 （連続送信）

⑤1～2分後に通知メールが来ていることを確認します。またCloudWatchの画面でもアラームが確認できます。Postmanの「Console log」を確認すると「orderId」がランダムに生成されていることが確認できます。